



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

ANALIZA PARADIGMELOR DE ANOMALY DETECTION PE DIFERITE SETURI DE DATE

Absolvent

Savin Radu-Andrei

Coordonator științific

Paul Irofti

București, iunie 2021

Rezumat

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce vitae eros sit amet sem ornare varius. Duis eget felis eget risus posuere luctus. Integer odio metus, eleifend at nunc vitae, rutrum fermentum leo. Quisque rutrum vitae risus nec porta. Nunc eu orci euismod, ornare risus at, accumsan augue. Ut tincidunt pharetra convallis. Maecenas ut pretium ex. Morbi tellus dui, viverra quis augue at, tincidunt hendrerit orci. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam quis sollicitudin nunc. Sed sollicitudin purus dapibus mi fringilla, nec tincidunt nunc eleifend. Nam ut molestie erat. Integer eros dolor, viverra quis massa at, auctor.

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce vitae eros sit amet sem ornare varius. Duis eget felis eget risus posuere luctus. Integer odio metus, eleifend at nunc vitae, rutrum fermentum leo. Quisque rutrum vitae risus nec porta. Nunc eu orci euismod, ornare risus at, accumsan augue. Ut tincidunt pharetra convallis. Maecenas ut pretium ex. Morbi tellus dui, viverra quis augue at, tincidunt hendrerit orci. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam quis sollicitudin nunc. Sed sollicitudin purus dapibus mi fringilla, nec tincidunt nunc eleifend. Nam ut molestie erat. Integer eros dolor, viverra quis massa at, auctor.

Cuprins

1	Introducere	6
1.1	Context și Motivație	6
1.2	Structura Lucrării	7
2	Preliminarii	8
2.1	Proiectarea și Abordarea Cercetării	8
2.2	Algoritmi de Detecție a Anomaliilor	9
2.2.1	One-Class SVM	9
2.2.2	Isolation Forest	9
2.2.3	Autoencoder Feedforward (MLP)	10
2.2.4	Autoencoder LSTM	10
2.2.5	LSTM Predictiv	10
2.2.6	Autoencodere Bazate pe Transformer	11
2.3	Seturile de Date	12
2.3.1	Credit Card Fraud Detection (Setul de Date A)	12
2.3.2	Yahoo Webscope S5 (Setul de Date B)	12
2.3.3	CIC-IDS2017 (Setul de Date C)	12
3	Methods and Experimental Setup	14
3.1	Preprocesare	14
3.1.1	Preprocesare Tabelară (Credit Card)	14
3.1.2	Crearea de Ferestre pentru Serii de Timp (Yahoo S5)	15
3.1.3	Preprocesarea Fluxurilor de Rețea (CIC-IDS2017)	16
3.2	Evaluation Protocol	17
3.2.1	Metrics Reported	17
3.2.2	Threshold Selection	18
3.2.3	No Point Adjustment	18
4	Experimental Results	19
4.1	Credit Card Fraud Detection (Dataset A)	19
4.1.1	Overall Comparison	19

4.1.2	Discussion	19
4.2	Yahoo Webscope S5 (Dataset B)	21
4.2.1	A1 – Point Anomalies (Real Server Metrics)	21
4.2.2	A3 – Contextual/Seasonal Anomalies	21
4.2.3	A4 – Level-Shift Anomalies	22
4.2.4	Trivial Baseline Sanity Check (Yahoo Only)	23
4.2.5	Discussion	24
4.3	CIC-IDS2017 (Dataset C)	25
4.3.1	Overall Comparison	25
4.3.2	Per-Attack-Type Breakdown	28
4.3.3	Discussion	30
4.4	Exploratory Data Analysis	30
4.4.1	Credit Card Fraud Detection	31
4.4.2	Yahoo Webscope S5	32
4.4.3	CIC-IDS2017	33
4.5	Explainability	34
5	Explainability Case Studies	35
5.1	Reconstruction-Error Localization on Yahoo A3	35
5.1.1	Initial Assumption	35
5.1.2	Qualitative Check: Per-Timestep Error Heatmaps	36
5.1.3	Quantitative Verification: Gini Coefficient of Error Concentration	37
5.2	Pseudo-Sequence Reconstruction Error on Credit Card Fraud	38
5.2.1	Initial Assumption	39
5.2.2	Qualitative Check: Per-Feature-Position Error Heatmaps	39
5.2.3	Quantitative Verification: Column-Shuffle Feature-Identity Check	41
5.3	Disjoint Feature Regimes: AE+Context vs. Temporal IForest	42
5.3.1	Temporal IForest: Initial Assumption	42
5.3.2	Temporal IForest: Score Validity Under Last-Flow Labeling	42
5.3.3	Temporal IForest: SHAP Attribution Across the Window	43
5.3.4	AE+Context: SHAP Dominated by Host-Context Features	44
5.3.5	Building the Combined Score	46
5.3.6	Two Fusion Rules, Two Normalizations	46
5.3.7	Per-Class Coverage vs. Aggregate AP	47
6	Concluzii	49
A	Alegeri legate de Hiperparametri și Antrenare	50
A.1	Isolation Forest	50
A.2	One-Class SVM	51

A.3	Autoencoder Feedforward (MLP)	51
A.4	LSTM Predictiv	52
A.5	Autoencoder LSTM	52
A.6	Bottleneck Transformer Autoencoder	53
A.7	Causal Transformer	53

Capitolul 1

Introducere

1.1 Context și Motivație

Această lucrare este un studiu experimental în învățarea automată, mai precis în subdomeniul *detectiei de anomalii* (*anomaly detection*) – sarcina de a detecta puncte, evenimente sau structuri care deviază de la normalitate. Domeniul este unul larg și acoperă o varietate de probleme, dintre care în această lucrare sunt abordate: detecția fraudelor în tranzacții bancare, detecția atacurilor asupra rețelelor de calculatoare și detecția defecțiunilor în metricile monitorizate ale serverelor.

O primă distincție care trebuie făcută este cea dintre *outlier detection* și *novelty detection*. Prima presupune antrenarea unui model pe date care conțin deja anomalii, cu speranța de a le identifica ulterior, în timp ce a doua presupune antrenarea exclusiv pe date considerate normale și evaluarea capacității modelului de a detecta, la testare, devieri de la noțiunea de normalitate astfel învățată. Această lucrare adoptă abordarea de tip *novelty detection*, întrucât acesta este cadrul în care sunt implementate în practică cazurile de utilizare menționate mai sus (un model de detecție a fraudelor, de exemplu, nu este antrenat pe cazuri confirmate de fraudă). În plus, această alegere ajută la formularea scopului lucrării printr-o singură întrebare: în ce măsură modul în care un model reprezintă structura latentă a datelor considerate „normale” determină tipurile de anomalii pe care acesta le poate, sau nu, detecta?

Motivația, cât și scopul acestei lucrări sunt, așadar, căutarea unui răspuns la această întrebare și abordarea ulterioară a ipotezelor formate în urma rezultatelor experimentale.

Contribuția proprie a acestei lucrări constă, așadar, în:

1. un pipeline standardizat de preprocesare, antrenare și evaluare, aplicat uniform pe trei seturi de date diferite și pe până la treisprezece configurații de modele și seturi de caracteristici (*features*);
2. o analiză sistematică a performanței diferitelor paradigme în funcție de tipul setului de date; și

3. o serie de studii de caz de explicabilitate, menite să testeze direct ipoteze formulate în secțiunea de discuții a rezultatelor, folosind hărți termice ale erorii de reconstrucție, o măsură de concentrație bazată pe coeficientul Gini și valori de atribuire SHAP, în locul unor concluzii bazate exclusiv pe metrice agregate.

1.2 Structura Lucrării

Restul lucrării este organizat în cinci capitole.

Capitolul 2 (*Preliminarii*) introduce fundalul teoretic necesar pentru restul lucrării: detalii despre funcționarea fiecărui model testat, paradigma căreia îi aparține și seturile de date folosite.

Capitolul 3 (*Methods and Experimental Setup*) descrie pipeline-urile de preprocesare, alegerile de hiperparametri și de arhitectură făcute pentru fiecare model, precum și protocolul de evaluare – inclusiv metricile raportate și deciziile metodologice (de exemplu, neaplicarea *point adjustment*-ului) care influențează modul în care trebuie citite rezultatele din capitolul următor.

Capitolul 4 (*Experimental Results*) prezintă rezultatul fiecărui experiment, organizat pe seturi de date, împreună cu o secțiune de discuții pentru fiecare set de date, care interpretează rezultatele prin raportare la așteptările stabilite în capitolul de Preliminarii.

Capitolul 5 (*Explainability Case Studies*) revizitează ipoteze specifice ridicate în discuția din Capitolul 4 și le testează direct, cu ajutorul unor diagnostice țintite – hărți termice ale erorii de reconstrucție, o măsură de concentrație bazată pe coeficientul Gini și valori de atribuire SHAP – confirmând unele dintre ele, nuanțând altele și, într-un caz, infirmând explicația inițială.

Capitolul 6 (*Concluzii*) rezumă concluziile lucrării, discută limitările acesteia și schițează posibile direcții de dezvoltare ulterioară.

Capitolul 2

Preliminarii

2.1 Proiectarea și Abordarea Cercetării

Pentru a răspunde la întrebarea formulată mai sus, am propus următoarea abordare: în loc să optimizez o singură arhitectură pentru un singur set de date, am realizat o comparație de-a lungul a două axe simultan.

Prima axă este **paradigma**: cele șapte modele evaluate (§2.2) au fost alese special pentru a acoperi patru moduri structural distincte de a reprezenta cum arată „normalul” – o **graniță** trasată într-un spațiu indus de un kernel (One-Class SVM), o **partiționare** a spațiului de caracteristici prin diviziuni aleatoare (Isolation Forest), un **bottleneck de compresie** învățat, care trebuie să renunțe la orice, cu excepția structurii dominante a datelor normale (cele trei autoencodere bazate pe reconstrucție), și un model **predictiv** al modului în care datele normale evoluează pas cu pas (LSTM predictiv, Transformer cauzal). Fiecare paradigmă codifică o presupunere implicită diferită despre ce anume face un punct anormal, ceea ce ridică întrebarea centrală a acestei lucrări: care presupunere se potrivește cărui tip de anomalie.

A doua axă este **setul de date**: Credit Card Fraud, Yahoo Webscope S5 și CIC-IDS2017 (§2.3) au fost alese pentru a acoperi un spectru de structură temporală – de la un set de date pur tabelar, fără o ordonare semnificativă între rânduri, trecând prin serii de timp cu adevărat secvențiale, până la date de flux de rețea care admit atât o perspectivă plată, cât și una temporală asupra acelorași evenimente. Rularea fiecărei paradigme de model pe fiecare punct al acestui spectru este ceea ce face posibilă atribuirea unui rezultat dat *paradigmei*.

În următoarele secțiuni vor fi prezentate detaliile algoritmilor și seturilor de date alese.

2.2 Algoritmi de Detecție a Anomaliilor

2.2.1 One-Class SVM

One-Class SVM (formularea ν -SVM) învață o graniță în jurul datelor de antrenare, într-un spațiu de caracteristici (de obicei infinit-dimensional) indus de un kernel. Trei hiperparametri, fixați identic pe toate cele trei seturi de date (`kernel='rbf'`, `nu=0.01`, `gamma='scale'` – vezi §A), determină împreună forma acestei granițe:

- `kernel='rbf'` fixează spațiul de caracteristici propriu-zis: fiecare punct de antrenare generează o **clopot gaussian** neted în jurul său, iar granița finală este conturul închis format prin suprapunerea acestor clopote în jurul zonelor dense de date.
- `gamma='scale'` stabilește cât de largă este fiecare clopot gaussian, deci cât de strâns se poate curba granița în jurul structurii locale a datelor – o valoare mai mare permite o graniță mai strânsă, dar cu risc mai mare ca modelul să învețe zgomotul.
- `nu=0.01` controlează cât de strâns este forțată granița să încadreze datele de antrenare, limitând la $\sim 1\%$ fracția de puncte benigne pe care modelul are voie să le lase în afara ei.

La testare, modelul returnează distanța semnată față de granița învățată (pozitivă în interior, negativă în exterior); aceasta este negată astfel încât, în concordanță cu toate celelalte modele din acest proiect, *scor mai mare = mai anormal*.

2.2.2 Isolation Forest

Isolation Forest construiește un ansamblu de arbori (numărul lor dat de parametrul `n_estimators`) care împart fiecare recursiv spațiul de caracteristici pe baza unor caracteristici și puncte de divizare alese aleator, și folosește lungimea medie a căii necesare pentru a izola un punct (în propria sa frunză), calculată pe toți arborii, drept semnal de anomalie. Deoarece diviziunile sunt aleatoare, punctele ușor de separat de restul datelor – adică anomaliile, care tind să se afle în regiuni rare, de densitate scăzută – sunt izolate în mai puține diviziuni, obținând astfel o lungime medie a căii mai mică decât punctele benigne tipice. Modelul returnează lungimea medie negată a căii, care este negată încă o dată aici, astfel încât un scor mai mare să însemne o cale mai scurtă (adică mai anormal), păstrând convenția consistentă cu toate celelalte modele. Parametrul `contamination` este lăsat la valoarea implicită, `'auto'`: acest parametru ar seta în mod normal pragul intern de decizie al modelului pe baza unei fracții presupuse de anomalii în datele de antrenare, dar cum datele de antrenare aici sunt exclusiv benigne (nicio contaminare de luat în calcul), iar pragul de operare este ales extern prin baleierea curbei PR din §3.2, `contamination` nu are niciun efect asupra rezultatelor raportate – contează doar scorul de anomalie continuu.

2.2.3 Autoencoder Feedforward (MLP)

Autoencoder Feedforward este bazat pe reconstrucție: o pereche simetrică encoder/decoder de straturi complet conectate **Linear+ReLU** (cu dropout între straturile ascunse) comprimă vectorul de caracteristici de intrare printr-un bottleneck de dimensionalitate strict redusă, apoi îl extinde înapoi la dimensiunea originală, fiind antrenat integral pentru a minimiza eroarea medie pătratică de reconstrucție față de propria intrare. Deoarece bottleneck-ul este mai îngust decât intrarea, rețeaua nu poate învăța pur și simplu funcția identitate; este forțată să învețe o reprezentare comprimată care surprinde structura dominantă specifică datelor *benigne*. La testare, scorul de anomalie este eroarea medie pătratică per eșantion dintre intrare și reconstrucția sa: eșantioanele benigne, care seamănă cu distribuția de antrenare în jurul căreia a fost modelat bottleneck-ul, se reconstruiesc cu eroare mică, în timp ce eșantioanele anormale – pe care bottleneck-ul nu a fost niciodată antrenat să le reprezinte – se reconstruiesc slab.

2.2.4 Autoencoder LSTM

Autoencoder LSTM aplică același principiu de reconstrucție ca Autoencoder Feedforward, dar pentru intrări sub formă de secvențe: un strat LSTM funcționează ca encoder, consumând întreaga secvență de intrare și păstrând doar starea ascunsă finală (și starea de celulă) ca un singur vector de dimensiune fixă, rezumând întreaga secvență. Această stare ascunsă finală este apoi repetată identic la fiecare pas de timp, ca intrare pentru un al doilea strat LSTM care acționează ca decoder, ale cărui ieșiri per pas de timp sunt proiectate înapoi la dimensionalitatea originală a caracteristicilor printr-un strat liniar comun. Scorul de anomalie este, din nou, eroarea medie pătratică per eșantion, calculată pe toți pașii de timp și toate caracteristicile simultan, între secvența de intrare și reconstrucția sa.

2.2.5 LSTM Predictiv

LSTM Predictiv folosește același bloc recurent (un singur strat LSTM) ca Autoencoder LSTM de mai sus, dar cu un obiectiv de antrenare diferit: este antrenat să prezică pasul $t + 1$ din pașii $0..t$, simultan la fiecare poziție din secvență – modelul nu vede niciodată valoarea pe care este pus să o prezică. Din punct de vedere arhitectural, aceasta înseamnă că starea ascunsă a LSTM-ului la fiecare pas de timp este proiectată direct într-o predicție a pasului următor printr-un strat de ieșire liniar, fără separare encoder/decoder și fără vector de tip bottleneck: modelul consumă secvența o singură dată, iar fiecare stare ascunsă de-a lungul parcursului este folosită, nu doar cea finală. Scorul de anomalie este eroarea medie pătratică dintre valorile prezise și cele reale ale pasului următor, de-a lungul secvenței.

2.2.6 Autoencodere Bazate pe Transformer

Sunt folosite două variante de transformer, ambele înlocuind coloana vertebrală recurentă a Autoencoder LSTM cu un **TransformerEncoder** multi-strat (blocuri de auto-atenție + feedforward, codificare pozițională sinusoidală), diferind însă în modul în care este impusă constrângerea de bottleneck (reconstrucție) sau de cauzalitate (predicție):

- **Bottleneck Transformer Autoencoder** – bazat pe reconstrucție. Fiecare caracteristică/pas de timp de intrare este proiectat(ă) într-un token de dimensiune D_MODEL și trecut(ă) prin encoderul transformer cu auto-atenție completă (non-cauzală), astfel încât fiecare poziție poate atenționa liber orice altă poziție. Reprezentările rezultate per poziție sunt apoi *mediate global* (*global-average-pooled*) de-a lungul secvenței într-un singur vector de dimensiune D_MODEL , comprimat printr-un strat liniar la un $LATENT_DIM$ strict mai mic, apoi extins înapoi printr-un al doilea strat liniar la forma completă a secvenței aplatizate. Acest pas de comprimare-extindere este bottleneck-ul explicit de informație (analog stării ascunse finale din Autoencoder LSTM) care împiedică mecanismul de atenție să copieze pur și simplu intrarea în ieșire. Scor de anomalie: MSE per eșantion între intrare și reconstrucție, pe toate pozițiile deodată – întreaga secvență reconstruită este produsă printr-o singură trecere prin rețea, nu pas cu pas ca la un decoder recurent.
- **Causal Transformer** – predictiv-temporal, echivalentul bazat pe atenție al LSTM Predictiv. Este folosită aceeași structură de encoder, dar cu o mască de atenție cauzală (triunghiulară superior) aplicată, astfel încât poziția t poate atenționa doar pozițiile $0..t$; modelul este antrenat cu același obiectiv de predicție a pasului următor ca LSTM Predictiv, fără bottleneck de comprimare. Scor de anomalie: MSE per eșantion între valorile prezise și cele reale ale pasului următor.

Ambele variante folosesc deliberat un singur **TransformerEncoder**, în loc de structura completă encoder-decoder întâlnită frecvent în literatura despre transformere pentru serii de timp (de exemplu, encoderul cu atenție ProbSparse al Informer, cuplat cu un decoder generativ, sau proiectele bazate pe reconstrucție care cuplează un encoder Transformer cu un decoder convoluțional separat). Costul de antrenare și inferență crește odată cu numărul de straturi de atenție, pe toate seturile de date și modelele, iar un design de tip encoder-only păstrează acest cost, împreună cu numărul de parametri antrenabili, la aproximativ jumătate din ce ar necesita un model echivalent encoder-decoder, fără a sacrifica abilitatea de a face fie comprimare-și-reconstrucție, fie predicție cauzală.

Nu există o implementare Causal Transformer pentru setul de date credit card (§4).

2.3 Seturile de Date

2.3.1 Credit Card Fraud Detection (Setul de Date A)

Setul de date Kaggle Credit Card Fraud Detection conține 284,807 tranzacții bancare, dintre care 492 (0.172%) sunt frauduloase. Cele 30 de caracteristici prezente sunt: **Time** (secunde scurse de la prima tranzacție din set), 28 de caracteristici transformate prin PCA V1–V28 (anonimizate din motive de confidențialitate, deja decorelate și aproximativ zero-medie prin construcție), și **Amount** (valoarea brută a tranzacției, puternic asimetrică spre dreapta). **Time** reprezintă secunde scurse, nu o axă temporală structurată – nu există periodicitate, tendință sau noțiune de sesiune de exploatat. Acest lucru face din Setul de Date A cazul de control deliberat, „fără ordine temporală reală”.

Setul de date, împreună cu analiza exploratorie a comunității, este disponibil pe Kaggle.¹ Este disponibil și un notebook exploratoriu suplimentar, dedicat special gestionării datelor dezechilibrate și detecției anomaliilor pe acest set de date.², care a servit drept inspirație pentru o parte din scripturile acestui proiect.

2.3.2 Yahoo Webscope S5 (Setul de Date B)

Yahoo Webscope S5 constă din patru familii de benchmark (A1–A4), fiecare o colecție de mai multe serii de timp univariate independente, cu anomalii etichetate, dar fiecare construită pentru a izola un tip diferit de anomalie:

Benchmark	Serii	Origine	Tip de anomalie	Rata de anomalii
A1	67	Metrici reale de server	Valori punctuale aberante	1.76%
A2	100	Sintetic, o singură componentă periodică + tendință	Valori punctuale aberante	0.33%
A3	100	Sintetic, 3 componente de frecvență în interacțiune	Contextuală (deviație de fază)	0.56%
A4	100	Sintetic, aceeași structură, fără sezonality	Schimbări de nivel	0.50%

A1 este real și zgomotos (cicluri zilnice/săptămânale dezordonate); A2–A4 sunt sintetice și construite special astfel încât tipul de anomalie să fie controlat independent de forma seriei de bază. Acest lucru este ceea ce permite capitolului de Rezultate să atribuie diferențele de performanță *tipului de anomalie* (punctuală vs. contextuală vs. schimbare de nivel), și nu unor diferențe confundante între seturile de date.

2.3.3 CIC-IDS2017 (Setul de Date C)

CIC-IDS2017 (Canadian Institute for Cybersecurity) este un benchmark de detecție a intruziunilor în rețea, captat pe parcursul a cinci zile: ~2.8 milioane de fluxuri de rețea, fiecare descris de ~80 de caracteristici statistice extrase prin CICFlowMeter (durată,

¹<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

²<https://www.kaggle.com/code/janiobachmann/credit-fraud-dealing-with-imbalanced-datasets>

numărul și rata pachetelor/octeților, statistici ale timpului dintre sosiri, numărul de flag-uri TCP, dimensiuni de fereastră etc.). Luni este trafic 100% benign; 14 tipuri distincte de atacuri (varianțe DoS/DDoS, PortScan, forță brută, atacuri web, Bot, Infiltration, Heartbleed) sunt injectate de marți până vineri.

Se știe că distribuția brută/Kaggle a acestui set de date este substanțial coruptă (Engelen et al., WTMC 2021): aproximativ 26% din fluxuri sunt artefacte de tip „appendix” TCP, care moștenesc eticheta de atac a fluxului părinte fără a conține trafic de atac real, o mare parte din fluxurile etichetate nominal ca atac nu au încărcătură utilă reală (etichetate doar formal, nu pe bază de comportament), iar întreaga clasă DoS Hulk corespunde unei simulări de atac defecte, care nu se execută niciodată efectiv împotriva serverului țintă. Sunt prezente și coloane de identificare (**Flow ID**, IP sursă/destinație, **Timestamp**) care, dacă sunt păstrate, permit unui model să „învețe pe scurtătură” după adresa IP sau fereastra de captură, în loc de comportamentul traficului. Acest proiect folosește versiunea curățată și reextrasă a setului de date de la Engelen et al., în locul CSV-urilor originale Kaggle/UNB, și elimină explicit coloanele de identificare (vezi §3.1).

Fluxurile CIC-IDS2017 sunt agregate statistice la nivel de sesiune. Acest lucru face din acest set de date un caz de tranziție perfect: admite o perspectivă validă de tip plat/tabelar (un flux = un eșantion, analog setului de Date A), o perspectivă temporală validă (fluxuri grupate și ordonate per gazdă, analog setului de Date B).

Capitolul 3

Methods and Experimental Setup

Hiperparametrii și alegerile legate de antrenare, per model (optimizator, număr de epoci, lățimi ale straturilor și toate celelalte detalii de arhitectură, câte un tabel per model pentru cele patru variante de set de date/set de caracteristici), sunt prezentate în Anexa A.

3.1 Preprocesare

Două principii se aplică la toate cele trei pipeline-uri: În primul rând, fiecare model este antrenat exclusiv pe eșantioane benigne/normale (*novelty detection*): partiția de antrenare nu conține niciodată o anomalie, iar toate scalarele/normalizatoarele sunt *potrivite (fit)* doar pe partiția de antrenare, apoi aplicate neschimbate pe partiția de testare – astfel evităm probleme de data leakage. În al doilea rând, partiția de testare conține întotdeauna ambele clase, evaluate cu protocolul descris în §3.2.

3.1.1 Preprocesare Tabelară (Credit Card)

1. **Time** este eliminat (nu reprezintă informație semnificativă, întrucât este doar timpul de la prima tranzacție din setul de date).
2. Rândurile sunt împărțite pe clase: toate rândurile benigne (**Class=0**) sunt reunite, apoi împărțite 80/20 în antrenare și test-benign. Setul de antrenare este partiția de 80%, conținând doar rânduri benigne (~227,451 rânduri).
3. Setul de testare este format din restul de 20% rânduri benigne, plus *toate* cele 492 de rânduri de fraudă.
4. Scalarea este asimetrică, pe grupuri de caracteristici: **V1--V28** trec **nescalate** implicit (**V_SCALING = 'none'** – întrucât acestea sunt deja caracteristici PCA), cu **'standard'** încercat de asemenea în unele rulări, fără o diferență reală între modele. **Amount** este scalat independent cu un **RobustScaler** (bazat pe mediană/IQR),

ales pentru robustețea sa la coada dreaptă accentuată, urmând același raționament ca notebook-ul exploratoriu citat în §2.

3.1.2 Crearea de Ferestre pentru Serii de Timp (Yahoo S5)

1. Fiecare serie este împărțită 70/30 de-a lungul axei temporale (`TRAIN_FRAC=0.70`) – o împărțire temporală și nu o amestecare aleatoare. Nu vrem ca vreo informație din viitor să se scurgă în antrenare.
2. Un `MinMaxScaler` este potrivit *per serie*, folosind doar punctele neanormale din partiția de antrenare a acelei serii, apoi aplicat atât partiției de antrenare, cât și celei de testare ale aceleiași serii.
3. Se aplică o fereastră glisantă de lungime `WINDOW=64`: pas 4 pentru ferestrele de antrenare (subeșantionate, păstrându-se pentru antrenare doar ferestrele complet normale) și pas 1 pentru ferestrele de testare (dens – este evaluată fiecare poziție posibilă a ferestrei).
4. Eticheta unei ferestre urmează regula „oricare”: pozitivă dacă cel puțin unul dintre cei 64 de pași de timp este anormal. Aceasta este totuși o sursă de „estompare” a etichetelor – de exemplu, pe A1, o rată de anomalii la nivel de punct de 1.76% devine o rată la nivel de fereastră de $\sim 17\text{--}18\%$, întrucât un singur punct anormal contaminează până la 64 de ferestre suprapuse. O fereastră mai mică ar reduce această estompare, lucru care ar putea merita încercat în viitor.
5. Ferestrele din toate seriile unui benchmark (reamintesc că fiecare benchmark este alcătuit din mai multe serii univariate) sunt reunite în array-uri unice.

Comparație cu DeepAnt. DeepAnt (Munir et al., 2019) a fost folosit ca referință pentru evaluarea unui predictor neural al pasului următor pe Yahoo S5, deși mai multe alegeri metodologice diferă de cele ale acestui proiect:

- **Predicție many-to-one, nu many-to-many.** Predictorul propus de DeepAnt este o rețea convoluțională; varianta LSTM raportată de aceștia ca referință de comparație (două straturi suprapuse, de câte 32 de unități) este mai îngustă și mai puțin adâncă decât LSTM-ul de 64 de unități folosit aici pentru LSTM Predictiv. Mai important, ambele prezic doar un singur pas de timp următor dintr-o fereastră (many-to-one), în loc de fiecare pas $t+1$ din $0..t$ simultan, așa cum fac modelele predictive ale acestui proiect (§2).
- **Antrenare per serie vs. antrenare reunită.** DeepAnt antrenează și evaluează un model separat pentru fiecare serie de timp individuală (o împărțire 40/60 antrenare/test per serie, scorul F calculat per serie și apoi mediat per sub-benchmark),

ceea ce permite fiecărui model să se specializeze pe o singură frecvență. Acest proiect reunește în schimb ferestrele din toate cele ~ 100 de serii ale unui benchmark într-un singur model comun, antrenat pe o împărțire 70/30 – în principal din motive de simplitate a implementării.

3.1.3 Preprocesarea Fluxurilor de Rețea (CIC-IDS2017)

Există trei implementări pentru acest set de date, toate având la bază aceiași pași de curățare (eliminarea `Flow ID`/`Src IP`/`Dst IP`/`Timestamp`/`Src Port`; codificare one-hot pentru `Protocol`; înlocuirea `Inf` cu `NaN`; imputare cu mediana, calculată pe setul de antrenare; eliminarea coloanelor cu varianță zero, măsurată pe setul de antrenare; `MinMaxScaler` potrivit doar pe setul de antrenare), dar diferind prin unitatea de observație:

Plat Un rând = un flux. Setul de antrenare este format din toate fluxurile benigne din ziua de luni (371,749 rânduri); setul de testare conține toate fluxurile benigne de marți-vineri, plus toate fluxurile de atac reale (fluxurile -- `Attempted`, adică atacuri fără încărcătură observată, sunt excluse complet). Filtrarea coloanelor cu varianță zero pe datele de luni elimină cele trei coloane de flag `URG`, rămânând 77 din cele 80 de caracteristici originale.

Temporal Un rând = o fereastră de `WINDOW=16` fluxuri consecutive de la aceeași adresă IP sursă, sortate după timestamp. Această alegere de grupare – după IP sursă, nu după sesiunea individuală de flux (identificată printr-un 5-tuplu: IP sursă, IP destinație, port sursă, port destinație și protocol – cei cinci identificatori pe baza cărora `CICFlowMeter` definește o conexiune individuală) – corespunde cheii de grupare folosite și pentru calculul caracteristicilor de context de gazdă de mai jos. Ambele pipeline-uri tratează „o entitate” drept fluxul de activitate ordonat al unei gazde sursă, nu o conexiune 5-tuplu, întrucât comportamentul la nivel de gazdă (multe conexiuni scurte, în rafală) caracterizează atacuri precum `PortScan` sau `DoS`, mai degrabă decât structura internă a unei singure conexiuni. Împărțirea este pe zile (luni antrenare, marți-vineri testare), din același motiv ca la pipeline-ul plat.

Context de gazdă Aceeași curățare plată, per flux, ca mai sus, dar fiecare flux este suplimentat cu 9 caracteristici construite, calculate dintr-o fereastră temporală glisantă de 60 de secunde, per IP gazdă: numărul de fluxuri, numărul de porturi unice, rata de diversitate a porturilor, un indicator de port nou, precum și asimetria sursă/destinație a fiecăreia. Acestea sunt calculate cauzal (doar din activitatea *anterioară* fluxului curent) și nu necesită etichete de atac, urmând adaptarea nesupervizată a lui Raskovalov et al. (2024). Numărul final de caracteristici: $77 + 9 = 86$. Această variantă a fost introdusă

special pentru a rezolva o slăbiciune comună pipeline-urilor plat și temporal, pe clasa de atac PortScan (vezi §4).

3.2 Evaluation Protocol

At test time, each model produces a scalar anomaly score per sample – never a hard label directly – and this label is obtained by thresholding that score, following the procedure described below.

3.2.1 Metrics Reported

Every run produces a text report containing the following:

Metric	Notes
Precision / Recall / F1 (per class)	Via <code>sklearn.classification_report</code> , reported for both the normal and anomaly class, plus accuracy and macro/weighted averages.
ROC-AUC	Reported for completeness / comparability with prior literature; not the primary ranking metric under class imbalance.
Average Precision (PR-AUC)	Threshold-independent; the standard choice under severe class imbalance, since ROC-AUC can look deceptively good when negatives vastly outnumber positives.
Balanced Accuracy	$\frac{1}{2}(\text{sensitivity} + \text{specificity})$ at the selected operating threshold.
F1 (positive/attack class)	At the selected operating threshold.
Confusion matrix counts	Raw TP/TN/FP/FN at the selected threshold.

For CIC-IDS2017 specifically, `evaluate_per_class()` additionally reports Average Precision broken down per attack type (treating each attack class against all benign samples in isolation) – this is what supports the per-attack-type discussion in §4.

PR-AUC (Average Precision) is treated as the primary metric for model comparison throughout this thesis, rather than F1 or ROC-AUC. This follows Kim et al. (2022), who observe that pre-defining a decision threshold without access to the test set is impractical, and that existing time-series anomaly detection methods instead report F1 at whichever threshold happens to maximize it – making reported results threshold-dependent by construction. In their Discussion, they conclude that „additional metrics with the reduced

dependency such as AUROC or area under precision-recall (AUPR) curve will help in rigorous evaluation”, which is the justification adopted here for ranking models by PR-AUC rather than by threshold-dependent F1.

3.2.2 Threshold Selection

A single decision threshold is still chosen per model/dataset to report the secondary threshold-dependent metrics (F1, balanced accuracy, confusion matrix) discussed above: it scans all thresholds on the precision-recall curve and picks the one maximizing F_β with $\beta = 2$, weighting recall twice as heavily as precision. The exact value of β is not tuned – any $\beta > 1$ would express the same operationally reasonable preference for an anomaly/fraud-detection setting, where a missed anomaly is typically costlier than a false alarm; $\beta = 2$ was simply fixed as a representative choice.

This is an *oracle* threshold: it is selected using the test set’s own precision-recall curve, which would not be available at deployment time. This is acceptable here because the object of study is relative model/paradigm comparison at the best achievable operating point, not deployment readiness – and because PR-AUC, the primary metric, does not depend on threshold selection at all. The F2/F1/balanced-accuracy figures at the oracle threshold should be read as secondary, illustrative numbers.

3.2.3 No Point Adjustment

Many time-series anomaly detection papers apply *Point Adjustment* (PA): if any point within a contiguous ground-truth anomaly segment is flagged, the entire segment is retroactively counted as detected. Kim et al. (2022) show this can inflate F1 to near-1 even for random scores when anomaly segments are long, and that PA-adjusted F1 is nearly uncorrelated with the un-adjusted F1 (Pearson $r = -0.59$ on the SWaT dataset in their study). No PA is applied anywhere in this project: window/flow labels are fixed at preprocessing time and predictions are scored exactly as thresholded, with no retroactive adjustment.

Capitolul 4

Experimental Results

4.1 Credit Card Fraud Detection (Dataset A)

4.1.1 Overall Comparison

Table 4.1 summarises all six evaluated models, ranked by Average Precision (AP), the primary metric. No Causal Transformer variant used for this dataset.

Model	Paradigm	ROC-AUC	Avg. Precision	F1 (fraud)
MLP Autoencoder	Reconstruction	0.9649	0.7105	0.6992
Isolation Forest	Partition	0.9587	0.6725	0.6070
LSTM Autoencoder	Reconstruction (pseudo-seq.)	0.9476	0.6223	0.7362
OC-SVM	Boundary	0.9354	0.5959	0.6454
Transformer Bottleneck AE	Reconstruction (pseudo-seq.)	0.9583	0.5947	0.6137
Predictive LSTM	Predictive (pseudo-seq.)	0.9581	0.5190	0.5244

Tabela 4.1: Credit Card Fraud Detection – model comparison, ranked by Average Precision.

All six models reach ROC-AUC > 0.93 , but AP spans 0.519–0.711 $\rightarrow$ ROC-AUC is far less sensitive to class imbalance than AP and can look deceptively uniform across models that behave very differently once the positive class is rare.

4.1.2 Discussion

No genuine sequence exists in this dataset. Unlike Yahoo S5 or the CIC-IDS2017 temporal/context variants, transaction rows have no temporal or causal ordering across features: **Time** — seconds since the dataset’s first transaction, an arbitrary reference point — is dropped at preprocessing; the pipeline predates the sequential models, added later purely for cross-dataset comparability, so it was left unchanged, leaving each row as an independent 29-dimensional vector (**V1--V28** PCA components + **Amount**). The three ”temporal” architectures evaluated here (Predictive LSTM, LSTM AE, Transformer

Bottleneck AE) do not see a real sequence – they reshape each 29-feature row into an artificial sequence of 29 scalar steps $((N, 29) \rightarrow (N, 29, 1))$, one feature per step, in the order present in the .csv file.

Impact on results. Predictive LSTM is the weakest model (AP 0.519): its next-step objective assumes a causal relationship between consecutive steps, but adjacent PCA components are orthogonal/decorrelated by construction, so there is no real next-step signal to learn – reflected in its worst false-positive count (FP=566). LSTM AE holds up comparatively well (AP 0.622, best F1 of all evaluated models at 0.736, lowest FP at 174) - its encoder still compresses the full input into one summary hidden state before decoding, functioning similarly to the MLP AE’s bottleneck rather than depending on order alone. Transformer Bottleneck AE (AP 0.595) sits close to the classical models for the same reason – global average pooling into a single latent vector, does not only rely on positional coherence between adjacent PCA components. MLP Autoencoder, the only architecture that treats the 29 features as an ordinary flat vector with no artificial windowing, wins overall (AP 0.7105) – consistent with there being no sequence to exploit.

This is the mirror image of the CIC-IDS2017 finding that host-context features (encoding *real* temporal/behavioural information) dominate, and of the Yahoo finding that the predictive objective wins under genuine periodic structure. Here, imposing sequence structure where none exists is a handicap: results rank models roughly by how much each architecture depends on the artificial ordering being meaningful – MLP AE (none) > LSTM AE / Transformer Bottleneck (soft, pooled/compressed) > Predictive LSTM (hard, depends on order).

Paradigm-level notes. Reconstruction is the strongest paradigm overall once the pseudo-sequence handicap is accounted for (MLP AE, LSTM AE, and Transformer Bottleneck AE occupy three of the top four spots). Isolation Forest is the best non-reconstruction model (AP 0.6725) and the second-best model overall – notably beating OC-SVM and two of the three temporal architectures, echoing CIC-IDS2017’s finding that simple, low-assumption models remain competitive whenever imposed structure is a poor match for the data. OC-SVM (AP 0.5959) sits mid-table, with the second-highest false-positive count after Predictive LSTM. Predictive LSTM is the only representative of the predictive paradigm on this dataset and the weakest model overall, for the structural reason discussed above. Given this, a Causal Transformer variant was not trained for this dataset — there is no genuine sequence for any predictive architecture to exploit.

Sanity check. LSTM AE’s best-of-all F1 (0.7362) despite a middling AP (0.6223) is a reminder that F1 is computed at a single oracle F2-threshold and can favor a different operating point than the one maximising area under the PR curve; AP remains the

primary metric for cross-model ranking.

4.2 Yahoo Webscope S5 (Dataset B)

A2 (single clean periodic component + trend, point outliers) is excluded from the comparisons below: like A1, it contains only point anomalies rather than contextual or level-shift ones, so it would pose the same limited-discriminative-power challenge discussed for A1 without adding a new anomaly type to the comparison.

4.2.1 A1 – Point Anomalies (Real Server Metrics)

Table 4.2 summarises the five models evaluated on A1. Causal Transformer and Transformer Bottleneck AE were not run on this benchmark.

Model	Paradigm	ROC-AUC	Avg. Precision
MLP Autoencoder	Reconstruction	0.792	0.611
LSTM Autoencoder	Reconstruction	0.794	0.600
OC-SVM	Boundary	0.776	0.595
Predictive LSTM	Predictive	0.789	0.588
Isolation Forest	Partition	0.765	0.511

Tabela 4.2: Yahoo S5 A1 (point anomalies) – model comparison, ranked by Average Precision.

Discussion. All five models cluster tightly within AP 0.51–0.61: MLP Autoencoder is best (AP 0.611) and Isolation Forest worst (AP 0.511), but the gap is narrow enough that no architecture shows a structural advantage here. Point anomalies (single-timestep spikes) do not require temporal understanding to detect. The window-level label blur discussed in §3.1 (a 1.76% point-level anomaly rate becomes ~ 17 –18% at the window level) further limits all models roughly equally, since it dilutes the discriminative signal available to every architecture in the same way.

4.2.2 A3 – Contextual/Seasonal Anomalies

Table 4.3 summarises all seven models evaluated on A3.

Discussion. Results diverge sharply once anomalies become contextual (a value normal in absolute terms but wrong for its phase in the seasonal cycle) rather than amplitude-based. Isolation Forest and OC-SVM collapse toward chance (ROC-AUC 0.511 and 0.584): a flattened 64-step window carries no phase information, so distribution-based detection fails outright. LSTM Autoencoder collapses just as badly (ROC-AUC 0.566) despite

Model	Paradigm	ROC-AUC	Avg. Precision
Predictive LSTM	Predictive	0.868	0.812
Transformer Bottleneck AE	Reconstruction	0.762	0.663
Causal Transformer	Predictive	0.738	0.623
MLP Autoencoder	Reconstruction	0.765	0.507
LSTM Autoencoder	Reconstruction	0.566	0.392
OC-SVM	Boundary	0.584	0.376
Isolation Forest	Partition	0.511	0.331

Tabela 4.3: Yahoo S5 A3 (contextual/seasonal anomalies) – model comparison, ranked by Average Precision.

being a sequence model – its encoder compresses the whole seasonal window into a single hidden state, showing this is a compression-bottleneck failure rather than a weakness of recurrent architectures generally. In the same vein, Transformer Bottleneck AE trails the best model (AP 0.663) but is by far the strongest *reconstruction*-based architecture, well ahead of LSTM AE: global self-attention over the window likely recovers periodicity that the LSTM AE’s bottleneck destroys, which probably accounts for the gap. Predictive LSTM dominates and is the best model on any Yahoo benchmark (AP 0.812, ROC-AUC 0.868): its next-step objective directly exploits how predictable a seasonal series is, so an injected anomaly produces a clean spike in prediction error exactly where it violates the expected trajectory.

4.2.3 A4 – Level-Shift Anomalies

Table 4.4 summarises all seven models evaluated on A4.

Model	Paradigm	ROC-AUC	Avg. Precision
Predictive LSTM	Predictive	0.745	0.505
Causal Transformer	Predictive	0.695	0.436
MLP Autoencoder	Reconstruction	0.719	0.402
Transformer Bottleneck AE	Reconstruction	0.655	0.404
Isolation Forest	Partition	0.498	0.268
OC-SVM	Boundary	0.519	0.265
LSTM Autoencoder	Reconstruction	0.502	0.252

Tabela 4.4: Yahoo S5 A4 (level-shift anomalies) – model comparison, ranked by Average Precision.

Discussion. The series now model a system that experiences abrupt structural shifts. Every model degrades relative to A3, regardless of paradigm: windows from all ~ 100 series are pooled together without series identity, and each series sits at its own baseline level – a

window’s absolute value alone cannot tell a model whether it is looking at a genuine post-shift window or an ordinary window from another series whose normal baseline happens to be similar. Classical models bottom out entirely under this confound – Isolation Forest reaches ROC-AUC 0.498, effectively a coin flip, since a shifted window can fall in a region of the pooled feature space already dense with unrelated series’ normal windows. Reconstruction autoencoders (MLP AE, Transformer Bottleneck AE) are blunted for the same reason and fail to overtake Predictive LSTM here despite a step-shaped window intuitively being easier to flag by reconstruction error; both plateau around AP 0.40. LSTM Autoencoder additionally collapses outright (ROC-AUC 0.502), reproducing the same encoder-compression failure seen on A3. Predictive LSTM still leads (AP 0.505, ROC-AUC 0.745), though its margin over the field narrows sharply from A3: because it conditions each prediction on a series’ own recent history rather than the pooled baseline distribution, a shift still registers as a violation of local continuity regardless of the absolute level it occurs at, but it probably generates a sustained, noisier error signal over the transition rather than the clean seasonal-deviation spike it could more easily exploit on A3.

4.2.4 Trivial Baseline Sanity Check (Yahoo Only)

Following Kim et al. (2022), who recommend checking every trained model against at least one label-free baseline before attributing its score to genuine representation learning, the models trained on Yahoo S5 were also evaluated against such a baseline (their ”Case 2”): score each window by its raw L2 norm $\|w\|_2$, with no learned model at all. Because point anomalies in A1–A3 are amplitude spikes, a window containing one has elevated magnitude even without any representation learning. Any trained model that fails to exceed this baseline on PR-AUC is not adding value over raw signal amplitude – this check is what confirms that the gains seen on A3 are genuine and not a window-magnitude artifact.

On A3, the baseline itself scores ROC-AUC 0.508 and AP 0.348 – indistinguishable from random, as expected for contextual anomalies that are phase violations rather than amplitude spikes. The *margin* each trained model holds above this baseline is a more informative quantity than its raw AP: the baseline’s own AP (0.348) closely matches the $\sim 33\%$ positive-window prior on A3 (itself elevated by the window-level label blur discussed in §3.1), confirming the baseline is pure noise and that a model’s raw AP is partly a function of this inflated prior rather than of learned structure alone.

- **LSTM AE (+0.04 AP):** barely above noise – encoder compression discards phase, leaving the model effectively using magnitude alone.
- **MLP AE (+0.16 AP):** captures some window-shape statistics implicitly.

- **Transformers (+0.28–0.29 AP):** global attention over the 64-step window recovers periodicity directly, a large and meaningful margin.
- **Predictive LSTM (+0.47 AP):** more than doubles the baseline’s AP – the strongest evidence in this project that the *training objective* (predict the next step), not raw model capacity, is the decisive factor on contextual anomalies.

4.2.5 Discussion

Paradigm and architecture both matter. The A3 results form a 2×2 comparison of paradigm (reconstruction vs. predictive) against architecture (LSTM vs. Transformer): reconstruction scores 0.39 AP (LSTM AE) and 0.66 AP (Transformer Bottleneck), while predictive scores 0.81 AP (Predictive LSTM) and 0.62 AP (Causal Transformer). The predictive paradigm is necessary – both predictive models beat both reconstruction models – but not sufficient on its own: the 0.19 AP gap between Predictive LSTM and Causal Transformer, despite an identical training objective and identical training data, shows the LSTM architecture carries an additional advantage on seasonal data. The likely explanation is that the LSTM’s recurrent hidden state acts as a continuous phase tracker, accumulating a compact summary of where the signal sits in its seasonal cycle at every step, so a phase violation produces a clean error spike; the causal transformer can attend to any earlier position directly but must learn explicit attention patterns to recover phase, a harder problem for A3’s three interacting frequency components. On A4 this architectural gap shrinks to 0.07 AP, since a single abrupt transition does not require phase accumulation, so the LSTM’s sequential inductive bias matters less.

Per-anomaly-type summary. Point anomalies (A1) leave all models within a narrow 0.51–0.61 AP band – no temporal understanding is needed to flag an isolated spike. Seasonal anomalies (A3) are where the predictive paradigm’s advantage is clearest, and where architecture choice matters most within that paradigm. Change-points (A4) degrade every model, but Predictive LSTM still leads, and Causal Transformer still beats both reconstruction models – the predictive advantage shrinks with weaker, shorter-lived signal, but does not flip. Classical models (Isolation Forest, OC-SVM) and LSTM Autoencoder collapse on any anomaly type that requires contextual rather than absolute discrimination (A3, A4), while remaining competitive only on A1.

4.3 CIC-IDS2017 (Dataset C)

4.3.1 Overall Comparison

Before comparing models, Table 4.5 breaks down the pooled attack flows in the flat/context test set by class, since every AP figure reported below is computed against this entire pool at once and not per class. This includes **DoS Hulk** (158,469 flows, tied with PortScan for the largest attack class): every preprocessing script labels any non-benign, non-attempted flow as positive with no per-class filtering, so DoS Hulk flows are counted as attack samples in every model’s aggregate ROC-AUC/AP/F1 in Table 4.7. However, Engelen et al. (WTMC 2021) show the DoS Hulk attack tool used to generate this dataset is misconfigured – it sets the HTTP **Connection** header to **Close** instead of **Keep-Alive**, so the target web server terminates the connection immediately instead of being kept busy, rendering the attack ineffective by construction. Consequently, DoS Hulk is deliberately excluded from the *per-class* breakdown in §4.3.2. This exclusion applies only to the per-class reporting, not to the pooled evaluation: DoS Hulk’s ~158k flows remain part of the positive class that every aggregate metric in Table 4.7 is computed against.

Attack class	n (flows)	% of attack pool
PortScan	159,000	36.66%
DoS Hulk	158,469	36.54%
DDoS	95,000	21.91%
DoS GoldenEye	7,500	1.73%
DoS slowloris	4,000	0.92%
FTP-Patator	4,000	0.92%
SSH-Patator	3,000	0.69%
DoS Slowhttptest	1,700	0.39%
Bot	738	0.17%
Web Attack – Brute Force	151	0.03%
Infiltration	32	0.01%
Web Attack – XSS	27	0.01%
Web Attack – SQL Injection	12	<0.01%
Heartbleed	11	<0.01%

Tabela 4.5: CIC-IDS2017 – attack-class composition of the pooled attack flows in the flat/context test set. DoS Hulk contributes to every aggregate metric in Table 4.7 but is excluded from the per-class breakdown in §4.3.2 (see text).

PortScan, DoS Hulk, and DDoS jointly account for 95.1% of all attack flows in this pool – directly substantiating the claim, made repeatedly below, that aggregate AP on this dataset is effectively a proxy for performance on these three classes, and that the ten remaining classes (all under 2% individually, seven of them under 0.2%) have limited leverage over the model ranking in Table 4.7 regardless of how well or poorly a model

handles them individually. This is also why a per-attack breakdown is necessary at all (§4.3.2 below): a model could in principle top Table 4.7 while being essentially blind to every interpretable attack type except PortScan and DDoS.

Table 4.6 gives the analogous breakdown for the temporal pipeline’s windowed test set. DoS Hulk is included here for the same reason as above.

Attack class	n (windows)	% of attack pool
PortScan	159,160	36.42%
DoS Hulk	158,469	36.27%
DDoS	95,131	21.77%
DoS GoldenEye	7,575	1.73%
DoS slowloris	4,001	0.92%
FTP-Patator	3,966	0.91%
Bot	3,320	0.76%
SSH-Patator	2,980	0.68%
DoS Slowhttptest	1,742	0.40%
Infiltration	445	0.10%
Web Attack – Brute Force	151	0.03%
Web Attack – XSS	26	0.01%
Web Attack – SQL Injection	20	<0.01%
Heartbleed	19	<0.01%
BENIGN	1,282,611	–

Tabela 4.6: CIC-IDS2017 – attack-class composition of the pooled attack windows in the temporal test set (1,719,616 windows total, 25.41% attack rate).

Window counts track flat flow counts closely for dense, bursty classes (DoS Hulk: 158,469 vs. 158,469 flat; similarly PortScan, DDoS, the DoS/Patator classes) – sliding `STRIDE_TEST=1` across a long uninterrupted attack span relabels nearly every position as attack without adding windows beyond what the span already spans. Sparse, isolated classes inflate far more sharply – Bot $\sim 4.5\times$ ($738 \rightarrow 3,320$), Infiltration $\sim 14\times$ ($32 \rightarrow 445$) – since each isolated flow gets ”smeared” across up to `WINDOW=16` overlapping windows under the any-attack rule (§3.1). Table 4.7 summarises all thirteen model/feature-set combinations evaluated on CIC-IDS2017, ranked by Average Precision (AP), the primary metric. Attack prevalence in the test set is $\sim 25\%$, considerably less extreme than Credit Card’s 0.17% but still imbalanced enough that AP and ROC-AUC diverge for the weaker models.

Model	Feature set	Paradigm	ROC-AUC	Avg. Precision	Balanced Acc.	F1 (attack)
MLP Autoencoder	Context	Reconstruction	0.9860	0.9487	0.9548	0.9020
OC-SVM	Context	Boundary	0.9745	0.9396	0.9569	0.9229
Isolation Forest	Context	Partition	0.9785	0.9328	0.9270	0.8367
LSTM Autoencoder	Temporal	Reconstruction	0.8976	0.7955	0.7846	0.6249
MLP Autoencoder	Flat	Reconstruction	0.9189	0.7921	0.8022	0.6305
Isolation Forest	Flat	Partition	0.9049	0.7612	0.8228	0.6639
OC-SVM	Temporal	Boundary	0.8629	0.7816	0.7600	0.5888
MLP Autoencoder	Temporal	Reconstruction	0.8831	0.7509	0.7734	0.6125
Isolation Forest	Temporal	Partition	0.8438	0.7604	0.6841	0.5195
OC-SVM	Flat	Boundary	0.7336	0.6845	0.6121	0.4644
Bottleneck Transformer AE	Temporal	Reconstruction	0.8077	0.6539	0.7059	0.5423
Causal Transformer	Temporal	Predictive	0.7099	0.5449	0.5520	0.4320
Predictive LSTM	Temporal	Predictive	0.6126	0.4472	0.5475	0.4295

Tabela 4.7: CIC-IDS2017 – all model/feature-set combinations, ranked by Average Precision.

Host-context features dominate. The three context-augmented models occupy the top three positions by every metric, with AP ranging 0.933–0.949. The gap between the best context model (AE+Context, AP 0.949) and the best non-context model (Flat MLP AE, AP 0.792) exceeds the entire spread across the remaining ten models. Each of the three base architectures improves substantially when context features are added: IForest gains +0.172 AP (0.761 $\rightarrow$ 0.933), MLP AE gains +0.157 (0.792 $\rightarrow$ 0.949), and OC-SVM gains the most, +0.255 (0.685 $\rightarrow$ 0.940) – the model that was worst among the flat trio becomes roughly tied for best. This shows that better features are sometimes more important than the choice of paradigm: OC-SVM’s RBF kernel is the same kernel finding the same kind of hypersphere boundary either way, but the 9 added spatial dimensions give it a well-separated kernel problem where the flat 77-feature space previously left benign and PortScan regions heavily overlapping.

Temporal windowing alone does not help. For IForest and the MLP AE, grouping flows into src-IP-ordered windows (§3.1) without adding context features gives no consistent improvement over the flat view, and sometimes hurts: Flat IForest (AP 0.761) $\approx$ Temporal IForest (AP 0.760), while Flat AE (AP 0.792) $>$ Temporal AE (AP 0.751). This is a direct contrast with the host-context gain above: both modifications add information about flow *order/activity over time*, but only the aggregate context statistics translate into a usable decision signal for the two attack classes (DDoS, PortScan) that dominate the test set by volume. The temporal windows carry the same underlying activity information implicitly, spread across 16 arbitrarily-ordered flows, which is a harder representation for these architectures to exploit directly.

Reconstruction beats prediction. Among the temporal-feature models, reconstruction-based architectures (LSTM AE, Temporal AE, Temporal OC-SVM) all clearly outperform the two prediction-based ones (Predictive LSTM AP 0.447, Causal Transformer AP 0.545) – the reverse of the Yahoo A3 finding (§4). Both predictive models’ threshold sweeps land at an extreme operating point (FP > 1.1M out of 1.28M benign flows), consistent with their low ROC-AUC (0.61 and 0.71): the scores themselves are poorly discriminative, not merely mis-thresholded.

4.3.2 Per-Attack-Type Breakdown

Table 4.8 breaks down Average Precision by attack class for the three context models, the strongest family overall. Class sizes vary by four orders of magnitude (11 to 159,000 samples), which itself shapes what is detectable at all.

Attack class	n	IForest+Ctx	OC-SVM+Ctx	AE+Ctx
DDoS	95,000	0.982	0.998	0.994
PortScan	159,000	0.601	0.820	0.842
DoS GoldenEye	7,500	0.655	0.215	0.220
DoS slowloris	4,000	0.190	0.012	0.025
DoS Slowhttptest	1,700	0.112	0.007	0.013
Heartbleed	11	0.093	0.001	0.000
Infiltration	32	AP 0.03–0.14, no clear winner across any model		
Bot	738	<0.01 across all models, including context variants		
FTP-Patator	4,000	<0.01 across all models, including context variants		
SSH-Patator	3,000	<0.01 across all models, including context variants		
Web Attack subtypes	12–151	near-zero AP everywhere (Brute Force, SQL Injection, XSS)		

Tabela 4.8: CIC-IDS2017 – per-attack-class Average Precision for the three context-augmented models.

Consistently well-detected. Two attack classes stand out as reliably detectable across nearly every feature set:

- **DDoS** is the easiest class by a wide margin: all context models score AP > 0.98 (OC-SVM+Context reaches 0.998), and even the flat models exceed AP 0.77. The very high packet rate and tiny packet sizes characteristic of a flood produce an extreme outlier in both per-flow and host-context feature spaces – `ctx_max_flow_cnt` alone nearly isolates it.
- **PortScan** is the clearest demonstration of the context features’ value: every flat and temporal model scores AP 0.07–0.28, while every context model exceeds 0.60, with AE+Context reaching 0.842. A single short scan flow to an arbitrary port is unremarkable in isolation, but `ctx_max_port_cnt` (unique destination ports contacted by the source IP in the preceding 60 seconds) is unmistakable during a scan

- probably helping especially with this attack. IForest+Context is comparatively weaker here (0.601).

Mixed context response for application-layer DoS. Not every attack class responds to context features the same way – the three application-layer DoS classes split the context family rather than favouring it uniformly:

- **DoS GoldenEye** shows a clear split within the context family: IForest+Context reaches AP 0.655, while AE+Context and OC-SVM+Context both collapse to ~ 0.22 . GoldenEye floods a single HTTP endpoint, producing many flows to one destination port – high `ctx_max_flow_cnt` paired with very low `ctx_max_port_rate`. Isolation Forest isolates this specific combination directly; the autoencoder reconstructs it with moderate error spread across many feature dimensions, reducing its discriminative power for this attack specifically.
- **DoS Slowhttptest and DoS slowloris** show the same pattern, more muted: IForest+Context is the only context model with meaningful detection (AP 0.112 and 0.190), though it still falls far short of Temporal IForest’s AP 0.69/0.61 on the same classes. Slow HTTP attacks maintain many concurrent long-duration connections to a single port – the resulting context signature (elevated `ctx_max_flow_cnt`, near-zero `ctx_max_port_rate`) is a tight, isolated cluster that IForest’s isolation mechanism targets well, while AE+Context and OC-SVM+Context fail to exploit it, scoring near-zero. The 60-second context window is a partial mismatch here, since Slowloris connections last minutes and many flows fall outside it – Temporal IForest’s explicit per-host chronological ordering still provides the best detection for this attack family.

Largely undetectable regardless of feature set. FTP-Patator, SSH-Patator, Bot, and the three Web Attack subtypes all score $AP < 0.01$ across every model, context included. Credential brute-force over legitimate protocols and HTTP-level attacks (SQL injection, XSS) produce flow statistics and host-context activity indistinguishable from normal usage – detecting them would require failed-authentication counters or payload inspection, neither available in a flow-statistics feature set. Heartbleed (11 samples) and Infiltration (32 samples) are too small to draw strong conclusions from, though IForest+Context’s modest edge on Heartbleed (AP 0.093 vs. near-zero for the other two context models) is consistent with the same axis-aligned-isolation advantage seen on the slow-DoS classes.

4.3.3 Discussion

Host-context features are the single most impactful addition in this study. Appending 9 rolling-window per-host activity statistics (§3.1) raises AP from 0.79 to 0.95 across the board – a larger gain than any other modelling choice in this project, larger than the choice of paradigm, and larger than moving from a flat to a temporal view of the same data – driven primarily by solving the PortScan blind spot that every flat and temporal model shared. No single context model dominates every attack type, even though AE+Context wins on the aggregate AP that is driven by the two largest classes.

A practical ensemble. No single model or feature set covers every attack class on this dataset. Pairing AE+Context (strong on DDoS and PortScan, the two classes that dominate the test set by volume) with Temporal IForest (strong on the slow-DoS classes and, tentatively, Heartbleed) would cover the widest range of detectable attack classes within this feature set – a concrete illustration of why the per-attack breakdown in §4.3.2 matters more than any single aggregate ranking. This premise, and a combined-score ensemble built and evaluated against it, are tested directly in §5.3: the two models’ evidence is confirmed genuinely disjoint, but combining their scores turns out to trade PortScan detection quality for gains on the slow-DoS classes rather than covering both for free.

4.4 Exploratory Data Analysis

This section collects the exploratory plots underlying the dataset-specific claims made in §3.1 and throughout this chapter.

4.4.1 Credit Card Fraud Detection

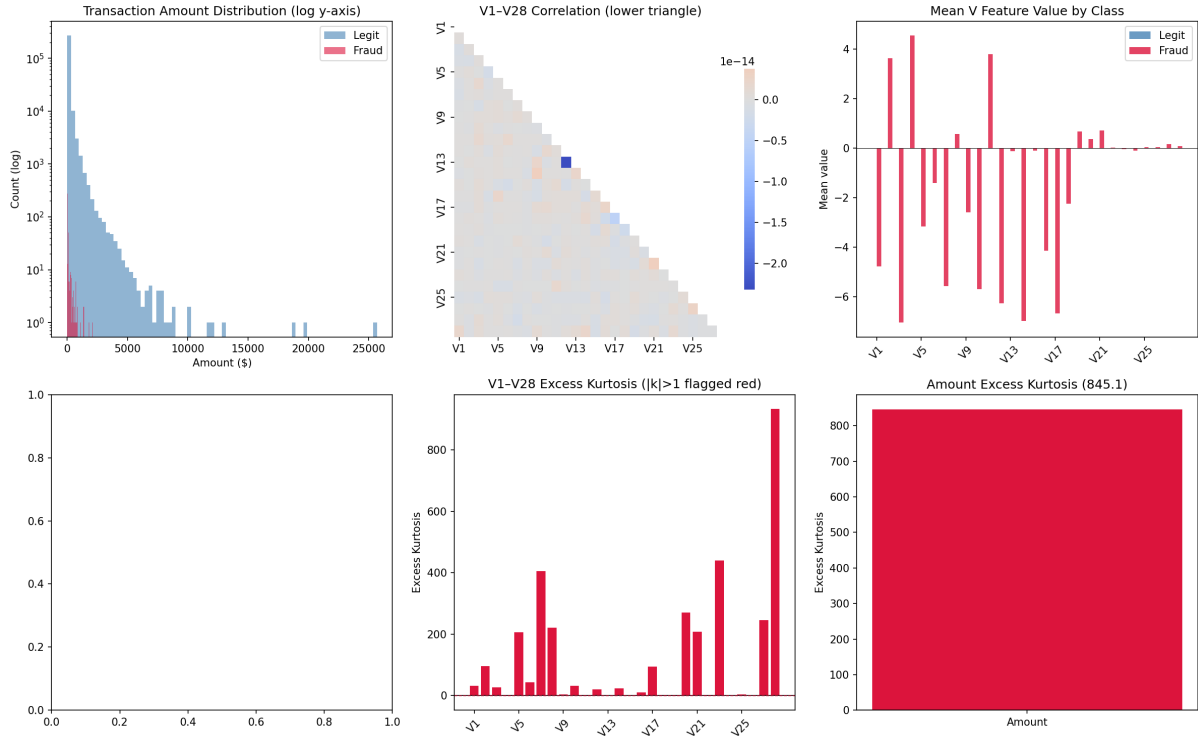


Figure 4.1: Credit Card Fraud Detection – transaction amount distribution, V1–V28 correlation structure, mean V-feature value by class, and excess kurtosis per feature (flagging the heavy-tailed PCA components discussed in the Datasets section).

4.4.2 Yahoo Webscope S5

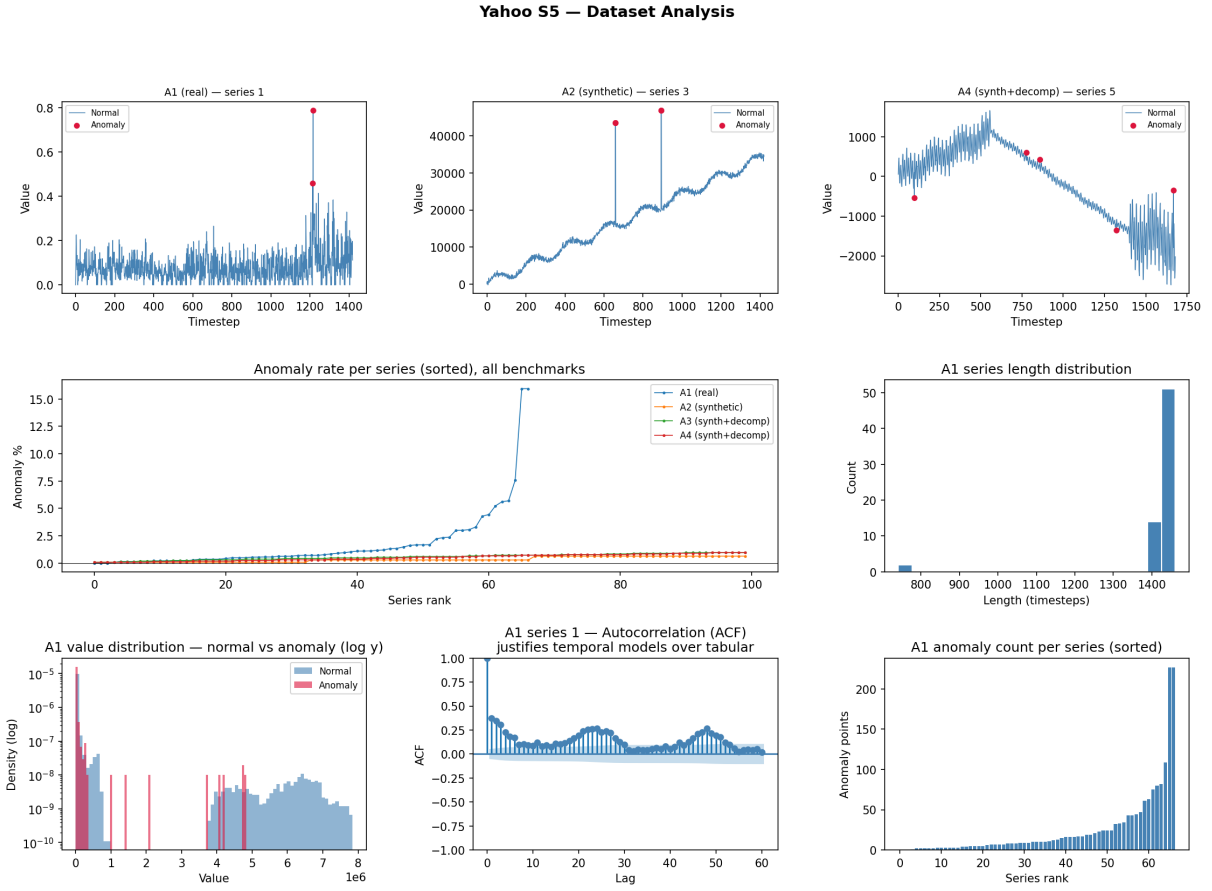


Figure 4.2: Yahoo S5 – example series per benchmark, per-series anomaly rate, A1 series length distribution, A1 value distribution for normal vs. anomalous points, A1 autocorrelation (motivating the use of temporal models over tabular ones), and A1 anomaly count per series.

4.4.3 CIC-IDS2017

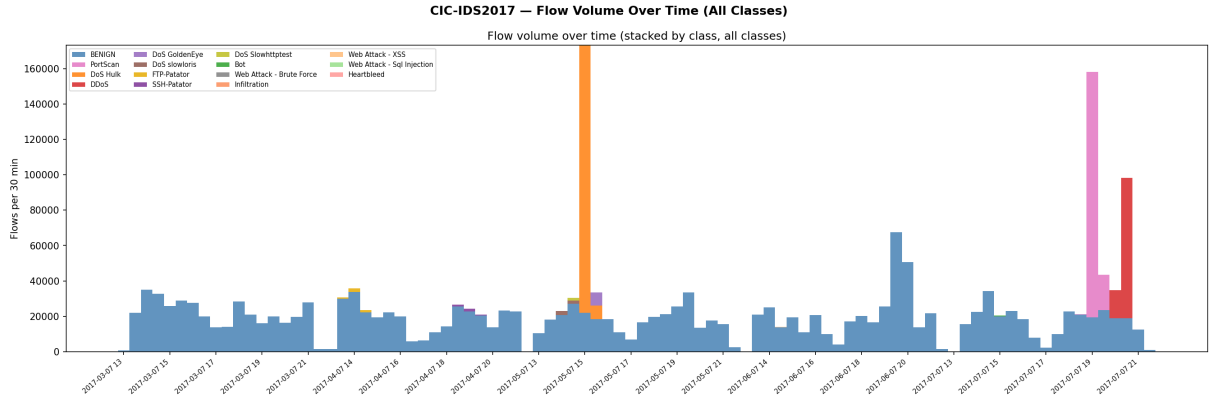


Figure 4.3: CIC-IDS2017 – flow volume over the capture week, stacked by class. BENIGN traffic dominates volume outside the concentrated attack windows visible on Tuesday–Friday.

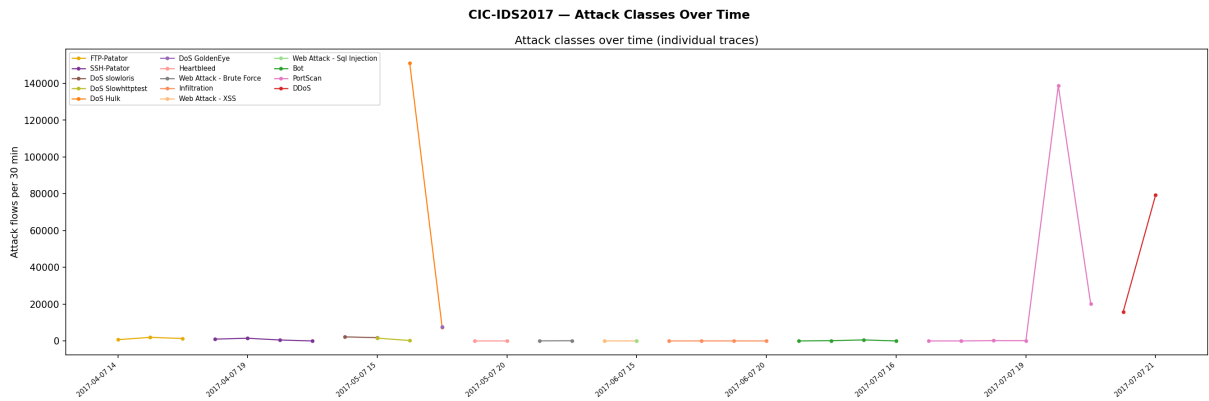


Figure 4.4: CIC-IDS2017 – attack classes over time, individual traces. Each attack occupies a short, distinct time window, confirming that attack classes do not overlap temporally within the capture.

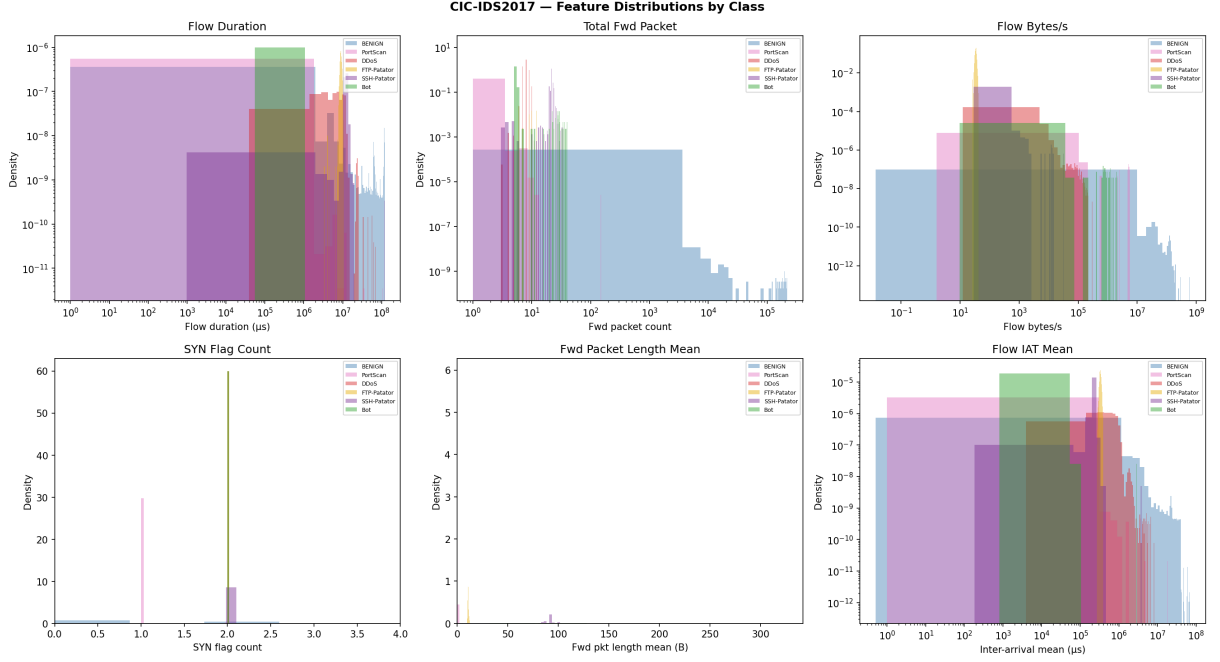


Figura 4.5: CIC-IDS2017 – per-class distributions of six flow-level features (flow duration, forward packet count, flow bytes/s, SYN flag count, forward packet length mean, inter-arrival mean) for BENIGN and five representative attack classes.

4.5 Explainability

Chapter 5 tests specific mechanistic claims made above (the LSTM/Transformer reconstruction gap on A3, the pseudo-sequence claim on Credit Card, and the disjoint-feature-regime premise behind the CIC-IDS2017 ensemble) with targeted SHAP and reconstruction-error diagnostics.

Capitolul 5

Explainability Case Studies

This chapter tests specific mechanistic claims made in Chapter 4 with targeted diagnostics, rather than relying on the aggregate metrics alone to support the mechanisms offered there.

5.1 Reconstruction-Error Localization on Yahoo A3

The A3 discussion in Chapter 4 attributes the Transformer Bottleneck Autoencoder’s lead over the LSTM Autoencoder (AP 0.663 vs. 0.392, ROC-AUC 0.813 vs. 0.566) to a mechanistic difference between the two bottlenecks: the LSTM Autoencoder compresses an entire 64-step window into a single final hidden state, which is hypothesized to destroy the window’s seasonal phase information and produce reconstruction error that is smeared diffusely across all timesteps; the Transformer’s global self-attention, by contrast, is hypothesized to preserve enough positional information to localize its reconstruction error sharply at the specific timestep where the window’s phase is violated. This is a plausible mechanism, but as stated it is inferred from the aggregate metrics alone – neither model’s per-timestep error was actually inspected. This section tests it directly.

5.1.1 Initial Assumption

The hypothesis under test makes an explicit, checkable prediction: for anomalous A3 windows, the LSTM Autoencoder’s per-timestep squared reconstruction error should look approximately *uniform* across the window’s 64 positions, while the Transformer’s per-timestep error should show a sharp, localized peak at the position where the seasonal pattern is violated. If instead both models produce similarly shaped error profiles, the AP/ROC-AUC gap would need a different explanation than the one offered.

5.1.2 Qualitative Check: Per-Timestep Error Heatmaps

Both trained models were reconstructed on every anomalous A3 test window, and the per-timestep squared error was plotted as a heatmap (one row per window, one column per of the 64 timesteps), with a random subsample of 300 windows used to keep the rendered image legible. Figures 5.1 and 5.2 show the result.

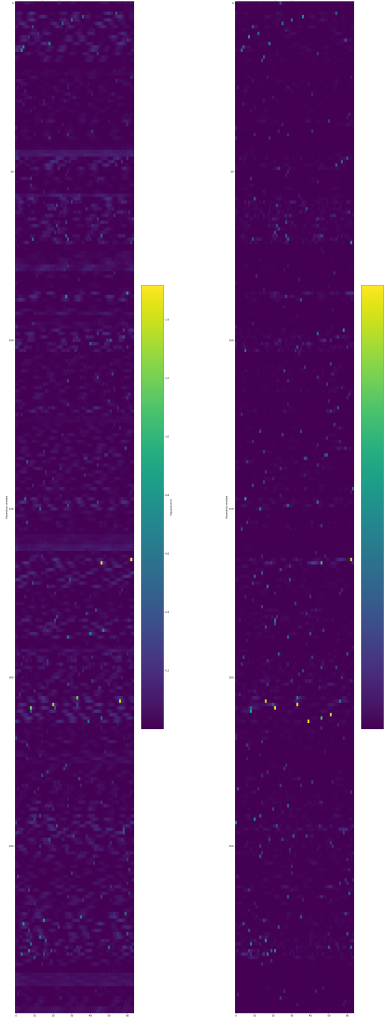


Figura 5.1: LSTM Autoencoder – per-timestep squared reconstruction error on 300 randomly sampled anomalous A3 windows (one row per window, left). Elevated error appears as sparse, punctate points rather than as a uniform band, but a visibly larger share of rows carry a low-level elevated baseline across most of their width compared to the Transformer.

Figura 5.2: Transformer Bottleneck Autoencoder – per-timestep squared reconstruction error on the same 300 anomalous A3 windows, same row order and colour scale convention as Figure 5.1 (right). Error is visibly more sparse and confined to isolated timesteps.

The visual comparison is broadly consistent with the initial assumption – the Trans-

former’s error concentrates into fewer, brighter points, while the LSTM Autoencoder’s error is more often spread thinly across a window’s full width – but a heatmap read by eye is a weak instrument here: each panel is normalized to its own colour scale, and the LSTM Autoencoder’s error values are roughly double the Transformer’s in magnitude throughout, which alone would make its background look more saturated regardless of how concentrated the error actually is. A qualitative read of Figures 5.1–5.2 therefore *suggests* the phase-destruction mechanism but cannot confirm it.

5.1.3 Quantitative Verification: Gini Coefficient of Error Concentration

To separate genuine error concentration from an artifact of differing overall error magnitude, each anomalous window’s 64 per-timestep squared errors were summarized with a Gini coefficient – a standard inequality measure (originally used for income distributions) that is invariant to the absolute scale of its input. For a window’s 64 error values sorted ascending, a Gini of 0 means the error is spread perfectly evenly across every timestep, and a Gini of 1 means it is concentrated entirely at a single timestep; unlike a raw magnitude comparison, this metric isolates the *shape* of the error distribution within a window from its overall size.

For n per-timestep squared errors $x_1 \leq x_2 \leq \dots \leq x_n$ sorted ascending (here $n = 64$), the Gini coefficient is computed as

$$G = \frac{2 \sum_{i=1}^n i x_i}{n \sum_{i=1}^n x_i} - \frac{n+1}{n}. \quad (5.1)$$

Computed over the full population of 14,449 anomalous A3 windows, the two models separate clearly:

	Mean Gini	Median Gini
LSTM Autoencoder	0.566	0.606
Transformer Bottleneck Autoencoder	0.721	0.724

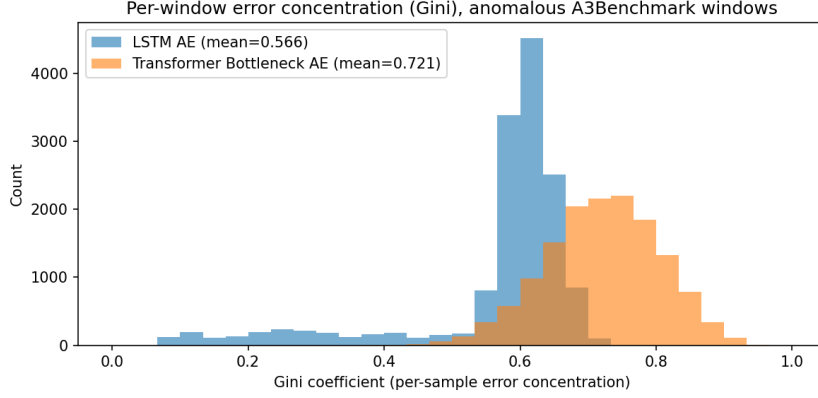


Figure 5.3: Distribution of per-window Gini coefficients across all 14,449 anomalous A3 windows. The Transformer’s distribution is unimodal and shifted toward high concentration (Gini > 0.6); the LSTM Autoencoder’s distribution is bimodal, with a majority clustered near the Transformer’s range but a distinct secondary population spread across low Gini values (error genuinely smeared across the window).

This confirms the initial assumption on average: the Transformer’s error is more concentrated than the LSTM Autoencoder’s across the full anomalous population, not merely in the handful of examples a heatmap happens to display. But the distribution in Figure 5.3 reveals a detail the aggregate means alone would miss – the LSTM Autoencoder’s Gini values are *bimodal*, not uniformly shifted down. A majority of its windows cluster in essentially the same high-concentration range as the Transformer’s (Gini ≈ 0.6 , comparable localization ability), while a distinct secondary population of windows sits at low Gini (< 0.4), where the error genuinely is smeared close to evenly across the full window. This indicates the phase-destruction mechanism is not a uniform property of every LSTM Autoencoder reconstruction, but a failure mode affecting a specific subset of windows – and it is this subpopulation, not a general degradation across all anomalous windows, that is responsible for dragging the LSTM Autoencoder’s aggregate AP and ROC-AUC below the Transformer’s, since on the remaining majority of windows the two models localize error comparably well.

5.2 Pseudo-Sequence Reconstruction Error on Credit Card Fraud

The Credit Card Discussion in Chapter 4 argues that the three “temporal” architectures (Predictive LSTM, LSTM Autoencoder, Transformer Bottleneck Autoencoder) do not see a genuine sequence: each reshapes a row’s 29 independent, decorrelated PCA features into an artificial sequence of 29 scalar steps, in whatever order the columns happen to appear in the source CSV. Predictive LSTM’s poor performance (AP 0.519) is attributed

directly to this – “adjacent PCA components are orthogonal/decorrelated by construction, so there is no real next-step signal to learn.” This section tests the reconstruction-based half of that claim (LSTM Autoencoder, Transformer Bottleneck Autoencoder) with the same per-position error diagnostic used in §5.1.

5.2.1 Initial Assumption

If the 29 column positions genuinely carry no meaningful order, per-position reconstruction error on fraud samples should look like unstructured noise: no column should consistently accumulate more error than any other across the fraud population, for either model. A structured, repeated pattern – specific columns lighting up across many fraud samples – would complicate that picture and call for a follow-up explanation.

5.2.2 Qualitative Check: Per-Feature-Position Error Heatmaps

Both trained models were reconstructed on the full fraud test population (492 samples), and the per-column squared error was plotted as a heatmap (one row per sample, one column per of the 29 feature positions), with a random subsample of 300 fraud samples used to keep the rendered image legible. Figures 5.4 and 5.5 show the result.

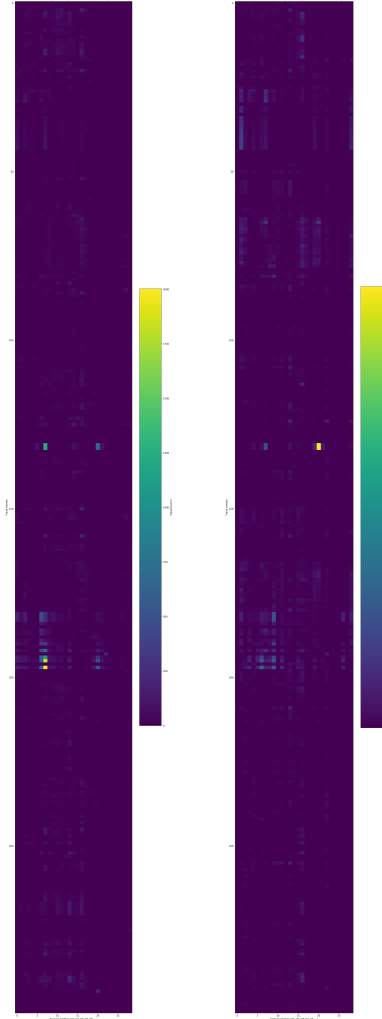


Figura 5.4: LSTM Autoencoder – per-feature-position squared reconstruction error on 300 randomly sampled fraud transactions (one row per sample, columns V1–V28 then Amount, left).

Figura 5.5: Transformer Bottleneck Autoencoder – per-feature-position squared reconstruction error on the same 300 fraud transactions, same row order and column convention as Figure 5.4 (right).

Contrary to the initial assumption, both heatmaps show a visibly structured, repeated pattern: a handful of column positions accumulate elevated error across many fraud rows, rather than error being scattered evenly across all 29 positions. This does not by itself contradict the "no real sequence" claim – it does not show that the models exploit *order*, only that error is not uniformly distributed across *columns*. Two distinct explanations are consistent with a structured heatmap: either specific PCA features are simply harder to reconstruct on fraud rows regardless of where they sit in the row (a feature-*identity* effect, fully compatible with "no real sequence"), or the models are exploiting some incidental adjacency between specific neighbouring columns in this one fixed ordering (a

position effect, which would nuance but not overturn the claim, since that adjacency is still arbitrary and non-causal). The heatmap alone cannot distinguish these.

5.2.3 Quantitative Verification: Column-Shuffle Feature-Identity Check

To separate the two explanations, the same 492 fraud rows were re-run through both models with a fixed random permutation of the 29 columns applied first (seed 42), and the per-column error was re-ranked. If the same underlying features dominate the top-error positions in both the original and the shuffled column order, the effect is feature-identity-driven; if overall error also changes substantially after shuffling, position/adjacency plays a role too.

	Overlap (top 10, orig. vs. shuffled)	Top-error magnitude, orig. $\rightarrow$ shuffled	Stable top features (in both orderings)
LSTM Autoencoder	6/10	47.7 $\rightarrow$ 71.7	V1, V3, V7, V8, V14, V17
Transformer Bottleneck AE	6/10	12.9 $\rightarrow$ 136.8	V2, V3, V7, V8, V14, V17

Both effects are present. Six of the top 10 highest-error features are the same before and after shuffling, for both models – well above the ~ 3 –4 expected in common between two independent random top-10 draws out of 29 features under a pure-noise null. **V14 and V17 are stable top accumulators for both models under both orderings**, and both are independently singled out by the community exploratory notebook cited in Chapter 3 as its two strongest negative correlates with the fraud label – an external check that these two features genuinely carry fraud-relevant signal, not an artifact of this experiment. At the same time, overall error rose substantially after shuffling for both models (LSTM Autoencoder’s top value: 47.7 $\rightarrow$ 71.7; Transformer’s: 12.9 $\rightarrow$ 136.8), which only makes sense if both encoders were also leaning on some incidental adjacency between specific neighbouring columns in the fixed, original CSV order.

Taken together, this confirms the core claim with a refinement the aggregate metrics could not surface on their own: the reconstruction-based architectures did learn to depend on the one fixed, arbitrary column ordering supplied at preprocessing time, beyond what a pure feature-identity effect would predict. The fact that error went up means the models had implicitly learned something that depended on which value sat next to which in the specific column arrangement the CSV happens to use, even though that adjacency is meaningless (it’s just PCA-component arbitrary order) - not evidence of a hidden causal sequence in the data, but more like the models overfit to an arbitrary but fixed layout

5.3 Disjoint Feature Regimes: AE+Context vs. Temporal IForest

The CIC-IDS2017 Discussion in Chapter 4 proposes pairing AE+Context (strong on DDoS and PortScan) with Temporal IForest (strong on the slow-DoS classes and, tentatively, Heartbleed) into a practical ensemble, since the two operate on disjoint feature regimes – host-context features versus a 16-flow temporal window. That argument rests on two separate premises, one per model: that Temporal IForest’s score is genuinely informed by the window’s temporal structure rather than by one flow’s static feature values sitting inside a wider, functionally inert vector; and that AE+Context’s score is genuinely informed by the host-context features unique to it, rather than by the same flow-level features Temporal IForest already has access to. Both are tested in turn below.

5.3.1 Temporal IForest: Initial Assumption

If Temporal IForest is genuinely exploiting cross-timestep structure, its per-window score should remain informative about the *last* flow’s own label – the flow that would be paired with AE+Context’s per-flow score in the ensemble – rather than only about whether *some* flow within the 16-flow window is an attack. Its SHAP attribution should also not collapse onto that single last flow; if it does, the window is decorative and the “disjoint feature regime” argument for the ensemble does not hold.

5.3.2 Temporal IForest: Score Validity Under Last-Flow Labeling

The originally showed per-class AP figures use an “any-of-16” window label: a window is positive if any of its 16 flows is an attack, tagged with the majority attack type. That is more lenient than the per-flow labeling AE+Context is scored against. Every existing test window was relabeled by its *last* flow’s own true label instead, and per-class AP recomputed against the unchanged, already-trained model and its existing scores.

	AP (any-of-16)	AP (last-flow)	<i>n</i> (last-flow)
DDoS	0.9518	0.9515	95,123
DoS GoldenEye	0.6278	0.6265	7,567
DoS Slowhttptest	0.6851	0.6849	1,742
DoS slowloris	0.6091	0.6077	4,001
Heartbleed	0.5927	0.2892	11

We see AP is essentially unchanged for all four target classes under the stricter labeling, confirming the score is informative about the last flow’s own status, not an artifact of the

lenient window label. Heartbleed drops sharply, but on only 11 true flows – its entire test population – so it is excluded from the ensemble and treated as tentative, consistent with its hedged treatment in Chapter 4.

5.3.3 Temporal IForest: SHAP Attribution Across the Window

Temporal IForest is trained on flattened 16-flow windows, so `TreeExplainer`’s SHAP values on the flattened input decompose cleanly back into a (window, timestep, feature) cube. For each of the four classes, SHAP was computed on 200 windows; attribution magnitude was summed over features per timestep to check for concentration on the final timestep, and the top contributing features were plotted across all 16 timesteps to check whether the same feature carries consistent-sign attribution throughout the window rather than only near the end.

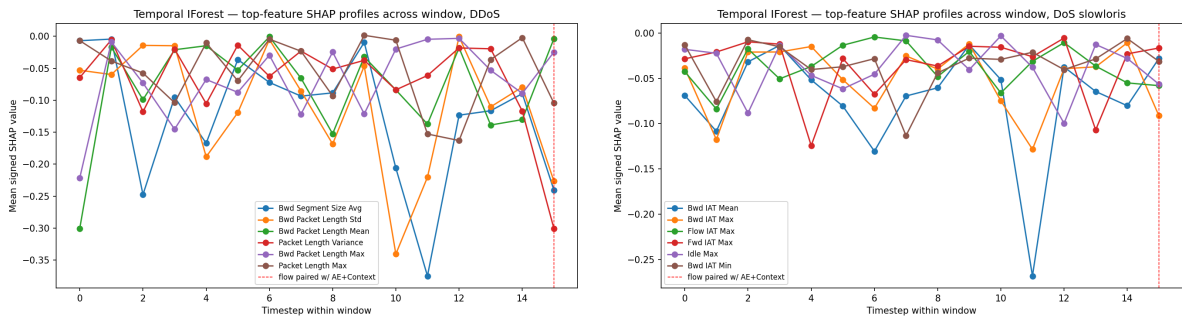


Figure 5.6: Temporal IForest – mean SHAP value per timestep for the six most-attributed features, DDoS windows (left) and DoS slowloris windows (right). In both cases attribution is negative (anomalous), recurs at consistent sign across most of the 16 timesteps, and does not concentrate at the final flow (dashed red line) – the flow paired with AE+Context’s score in the proposed ensemble.

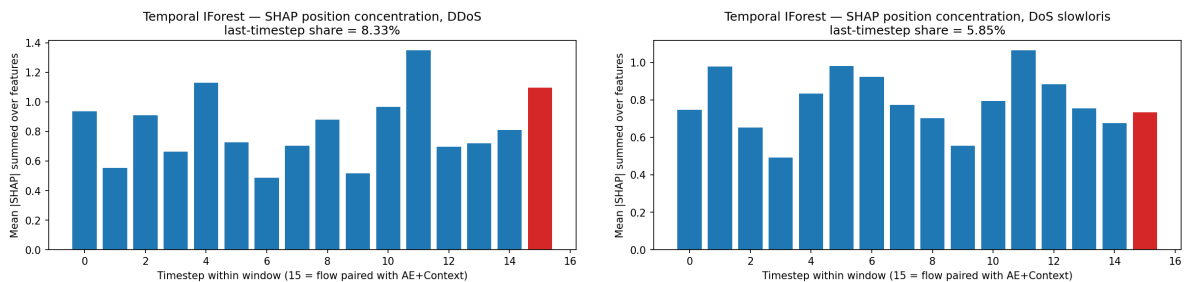


Figure 5.7: Temporal IForest – mean $|\text{SHAP}|$ summed over features, per timestep, DDoS windows (left) and DoS slowloris windows (right). The final timestep (red bar, 15) – the flow paired with AE+Context’s score in the proposed ensemble – carries no more attribution mass than any other position in the window, confirming the summary statistic reported below.

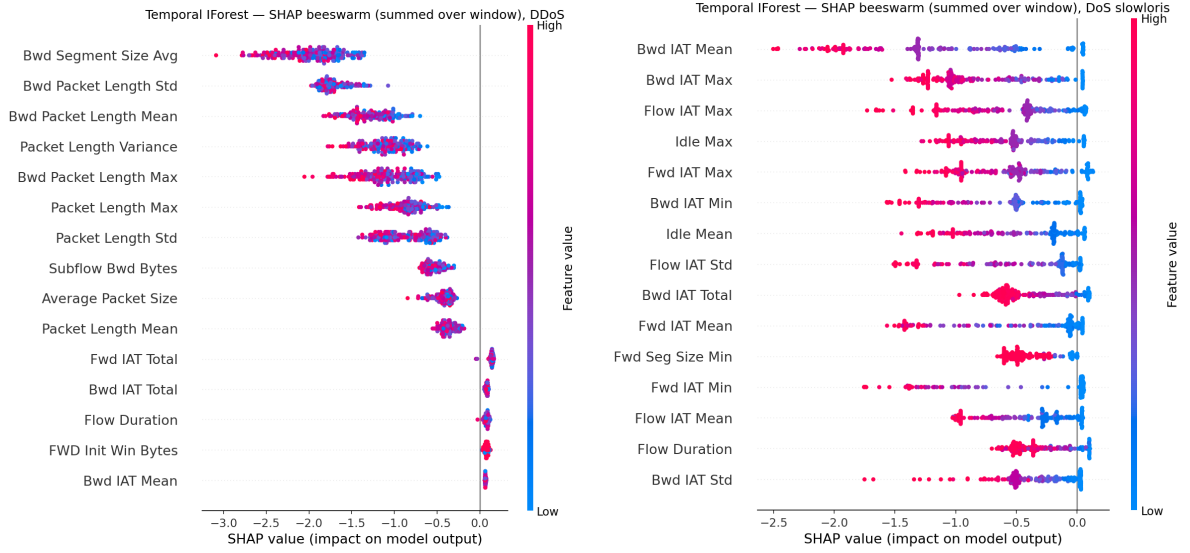


Figure 5.8: Temporal IForest – SHAP beeswarm per feature, DDoS windows (left) and DoS slowloris windows (right), collapsing the 16-timestep axis by summing each feature’s SHAP contribution across the window and coloring by that feature’s mean value across the window. DoS slowloris shows a clean monotonic relationship – high inter-arrival-time values drive strongly negative (anomalous) SHAP, low values push near zero – consistent with the slow, connection-holding behaviour the attack is named for. DDoS shows a noisier value-to-sign relationship for its backward-packet-length features, and a wider per-window spread than the mean-only view in Figure 5.6 suggests on its own.

The final timestep – the one paired with AE+Context in the ensemble – carries only 6–8% of total attribution mass across all four classes, close to the 6.25% uniform baseline for a 16-step window, rather than dominating it. The top features also match each attack’s known mechanism: DDoS and DoS GoldenEye attribution concentrates on backward-packet-length statistics (**Bwd Packet Length Std/Mean/Max**, **Packet Length Variance**), consistent with a volumetric flood; DoS Slowhttptest and DoS slowloris attribution concentrates on inter-arrival-time statistics (**Fwd/Bwd IAT Std/Max**, **Idle Mean/Max**), consistent with the slow, connection-holding behaviour these attacks are named for. This confirms the initial assumption: Temporal IForest’s advantage on these classes reflects genuinely disjoint temporal evidence, not a relabeled copy of one flow’s static features – supporting the “disjoint feature regime” premise behind pairing it with AE+Context.

5.3.4 AE+Context: SHAP Dominated by Host-Context Features

The mirror premise is that AE+Context’s advantage on its own strong classes – DDoS and PortScan, per Chapter 4 – traces to the 9 `ctx_*` host-context features unique to its

feature set, rather than to the same flow-level features Temporal IForest already sees. If instead AE+Context’s attribution spreads across ordinary flow features, its complementarity with Temporal IForest would be coincidental rather than a genuinely disjoint evidence source.

SHAP was computed with `GradientExplainer` on AE+Context’s reconstruction error, on 200 samples each of PortScan and DDoS, and the 15 features with the highest mean $|\text{SHAP}|$ were ranked for each class.

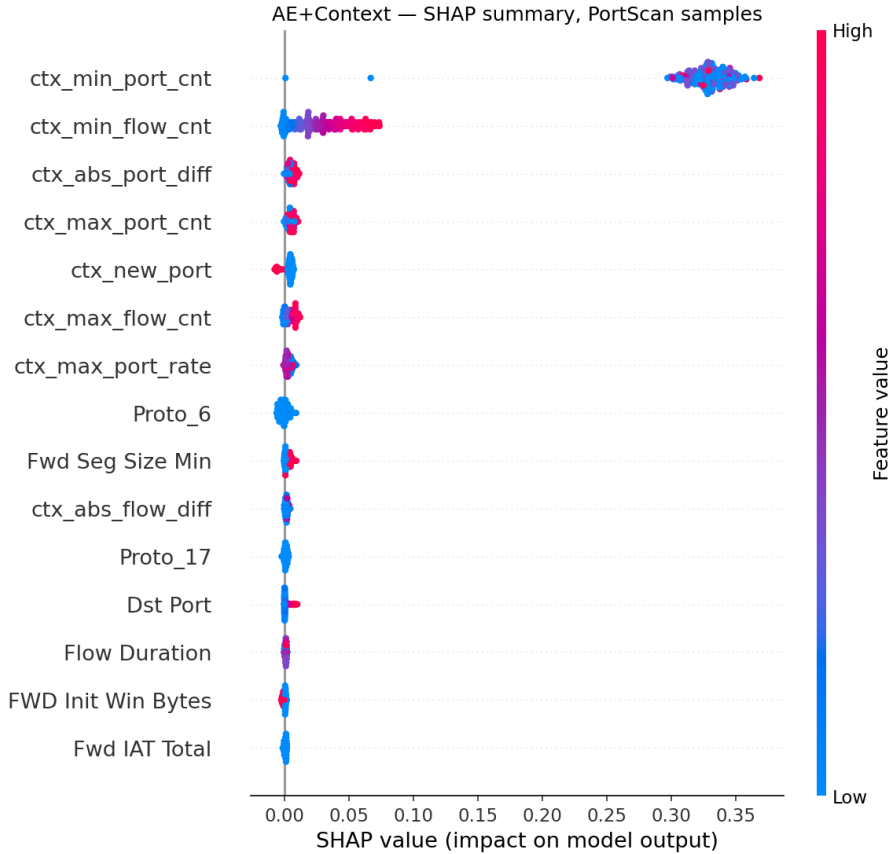


Figure 5.9: AE+Context – SHAP summary plot, PortScan samples. `ctx_min_port_cnt` dominates with a tight, uniformly positive-SHAP cluster well separated from every other feature; the remaining top rows are almost entirely `ctx_*` host-context features, with ordinary flow features only appearing near the bottom of the ranking.

	<code>ctx_*</code> features in top 15	Leading feature (mean $ \text{SHAP} $)
PortScan	8/15	<code>ctx_min_port_cnt</code> (0.324)
DDoS	7/15	<code>ctx_min_port_cnt</code> (0.234)

For both classes, `ctx_min_port_cnt` and `ctx_min_flow_cnt` dominate by roughly an order of magnitude over every ordinary flow feature: on PortScan, `ctx_min_port_cnt`’s attribution (0.324) is nearly $70\times$ the next-highest ordinary feature, `Fwd Seg Size Min` (0.0023); on DDoS it is over $6\times$ the leading ordinary feature, `Bwd Packet Length Std`

(0.035). Six of the nine `ctx_*` features appear in both classes’ top-15 lists, with only `ctx_min_port_rate` absent from both. This confirms the mirror premise: AE+Context’s advantage on these classes is driven almost entirely by host-context evidence unavailable to Temporal IForest, not by the ordinary flow features the two models share.

Taken together with §5.3.3’s result for Temporal IForest, the two halves of the disjoint-feature-regime premise both hold: Temporal IForest’s strong classes are explained by evidence spread across the 16-flow window (packet-length statistics for volumetric floods, inter-arrival-time statistics for slow-drip attacks), and AE+Context’s strong classes are explained by host-context features that exist only in its own feature set. The proposed ensemble’s complementary coverage is therefore backed by genuinely non-overlapping evidence between its two members, not by two rankings that merely happen to differ on which classes they cover.

5.3.5 Building the Combined Score

Confirming the disjoint-evidence premise does not by itself establish that combining the two scores works in practice. Realizing the ensemble sketched in Chapter 4 requires pairing, for every flow f in the test set, AE+Context’s own per-flow score $s_{AE}(f)$ with Temporal IForest’s score for the 16-flow window *ending* at f – the same causal, non-leaking choice validated in the previous subsection via last-flow labeling. Since the two preprocessing pipelines reorder flows differently before saving their respective test arrays (AE+Context sorts globally by timestamp; Temporal IForest sorts per source-IP host), the two score arrays are not aligned index-for-index and must be re-aligned by flow identity, recovering each flow’s original row position in both pipelines’ shared, pre-sort input.

Once aligned, the two raw scores cannot be combined directly: AE+Context’s score is a mean-squared reconstruction error with no fixed scale, and Temporal IForest’s score is a negated mean path length, and the two live on entirely different numeric ranges. Some normalization is required before either a weighted average or a max can be meaningful. Each model’s normalization statistics are computed on that same model’s own *benign training* scores only, never on test data, to avoid leaking test-set anomaly information into the fusion.

5.3.6 Two Fusion Rules, Two Normalizations

Two combination rules were tested. **Weighted soft voting** takes a convex combination of the two normalized scores, $s_{\text{soft}}(f) = w \cdot \hat{s}_{AE}(f) + (1 - w) \cdot \hat{s}_{IF}(f)$, swept over $w \in \{0.3, \dots, 0.7\}$. **Max-rule fusion** instead takes $s_{\text{max}}(f) = \max(\hat{s}_{AE}(f), \hat{s}_{IF}(f))$ – an OR-like combiner intended to match the disjoint-evidence story more directly: either model’s confidence alone should be enough to flag a flow.

The first normalization attempted, standard z -scoring against each model’s own benign training mean and standard deviation, turned out to be a dead end rather than a genuine test of either fusion rule. AE+Context’s reconstruction-error z -scores are extremely heavy-tailed (maximum $z \approx 2366$ on the test set, with the 99.9th percentile of *benign* flows alone already at $z \approx 227$), while Temporal IForest’s z -scores stay tightly bounded (maximum $z \approx 21$). Under z -score normalization, a handful of extreme AE+Context outliers dominate any linear combination or max regardless of the weight assigned to Temporal IForest: max-rule and every soft-vote weight tested reproduced AE+Context’s own per-class ranking almost exactly (e.g. GoldenEye AP stayed at 0.215–0.216 across all five weights, against AE+Context alone’s 0.215), rather than showing any influence from Temporal IForest’s much stronger 0.627 standalone score on that class. This is a normalization artifact, not evidence that fusion does not help – it does not actually test the ensemble hypothesis.

Replacing z -score with rank normalization – mapping each test score to its percentile within that model’s own benign training-score distribution, via the empirical CDF – removes this problem by construction: every normalized score is bounded to $[0, 1]$ regardless of how extreme the raw value is, so neither model’s tail can dominate the other’s by sheer magnitude. All results below use rank normalization.

5.3.7 Per-Class Coverage vs. Aggregate AP

	DDoS	PortScan	GoldenEye	Slowhttpstest	slowloris
AE+Context alone	0.882	0.924	0.292	0.013	0.025
Temporal IForest alone	0.956	0.123	0.611	0.578	0.560
Max-rule (rank)	0.882	0.923	0.292	0.060	0.083
Soft-vote $w = 0.4$ (rank)	0.977	0.228	0.801	0.205	0.319
Soft-vote $w = 0.7$ (rank)	0.977	0.358	0.821	0.140	0.245

Aggregate AP over the full row-aligned test set actually *favors* the variants that change the least: max-rule (aggregate AP 0.943) and AE+Context alone (0.941) both edge out soft-vote $w = 0.4$ (0.841), simply because DDoS and PortScan together account for over 250,000 of the roughly 434,000 positive flows in this evaluation, while GoldenEye, Slowhttpstest, and slowloris combined account for barely 13,000. An aggregate metric dominated by two classes’ raw volume is exactly the wrong lens for judging whether an ensemble motivated by *per-class coverage* actually delivers it, so §4.3.2’s per-class breakdown, not the aggregate table, is the right basis for evaluating this ensemble – consistent with how every other per-attack-type claim in this thesis is judged.

Read per-class, max-rule falls short of its own premise: because it is an OR over two scores, it inherits the *union* of both models’ false-positive tendencies rather than only their

true positives, so on GoldenEye/Slowhttpptest/slowloris it barely moves off AE+Context’s own weak scores ($0.292 \rightarrow 0.292$, $0.013 \rightarrow 0.060$, $0.025 \rightarrow 0.083$) despite Temporal IForest’s much stronger standalone numbers on exactly these classes. Soft voting shows the opposite failure mode on PortScan: because it averages rather than maxes, and Temporal IForest is actively poor on PortScan (0.123 , well below AE+Context’s 0.924) rather than merely uninformative, blending pulls PortScan’s ranking down substantially ($0.924 \rightarrow 0.228$ at $w = 0.4$) even while lifting GoldenEye from 0.292 to 0.801 .

Neither rule is a free win, but soft voting is the more defensible choice for a coverage-motivated ensemble: its *weakest* covered class (0.205 , Slowhttpptest, at $w = 0.4$) clears both standalone models’ own weakest classes – AE+Context’s 0.013 (Slowhttpptest) and Temporal IForest’s 0.123 (PortScan) – something neither standalone model nor max-rule fusion achieves. This is a genuine, if partial, realization of the ensemble idea from Chapter 4: rank-normalized soft voting measurably widens the floor of attack-class coverage, at the explicit cost of PortScan detection quality, rather than the free lunch the original aggregate-level framing implied. A deployment that must retain AE+Context’s near-complete PortScan coverage would instead need a class-aware combination rule (e.g. routing PortScan-like flows to AE+Context alone) rather than a single global weight – left as future work.

Capitolul 6

Concluzii

Anexa A

Alegeri legate de Hiperparametri și Antrenare

Două decizii principale se aplică uniform pentru toate modelele și seturile de date:

- **Optimizator.** Toate rețelele neuronale sunt antrenate cu Adam, la o rată de învățare fixă de 10^{-3} , uniform pentru fiecare model și set de date.
- **Fără căutare de hiperparametri.** Dacă un model a avut performanțe neașteptat de slabe, rata sa de învățare a fost verificată suplimentar cu $\{10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$ înainte de a trage vreo concluzie din acel rezultat.
- **Partiție de validare doar pentru monitorizare.** Fiecare rețea neuronală reține aleator 10% din datele de antrenare exclusiv pentru a trasa o curbă de pierdere (*loss*) de antrenare/validare per rulare, ca verificare că antrenarea decurge corect. Această partiție nu este niciodată folosită pentru oprire timpurie (*early stopping*) sau selecția modelului: antrenarea rulează întotdeauna pentru numărul fix de epoci de mai jos, iar rezultatele raportate pe setul de testare folosesc ponderile modelului din epoca finală.

A.1 Isolation Forest

	CIC flat	CIC context	Yahoo	Credit card
N_ESTIMATORS	150	150	150	150
MAX_SAMPLES	300,000	300,000	$\min(n, 50,000)$	200,000
Dimensiunea setului de antrenare	~371k	~371k	câteva mii	~227k
% din antrenare folosit	~81%	~81%	~100%	~88%
RANDOM_SEED	123	123	123	123

`MAX_SAMPLES` este singura diferență semnificativă și este setat cât mai ridicat posibil, practic, pentru fiecare set de date (acuratețea crește odată cu mai multe eșantioane per arbore, cu randamente descrescând; problema de „swamping” al arborilor nu apare aici, întrucât datele de antrenare sunt întotdeauna exclusiv benigne). Plafonul de 50.000 pentru Yahoo nu este atins niciodată în practică și este păstrat doar ca limită de siguranță.

A.2 One-Class SVM

	CIC flat	CIC context	Yahoo	Credit card
NU	0.01	0.01	0.01	0.01
KERNEL	rbf	rbf	rbf	rbf
GAMMA	scale	scale	scale	scale
SUBSAMPLE	50,000	50,000	niciunul (complet)	niciunul (complet)
% din antrenare folosit	~13.5%	~13.5%	~100%	~100%

Toți parametrii kernel/nu sunt uniformi pe seturile de date. Subeșantionarea este aplicată doar acolo unde setul de antrenare este suficient de mare încât un OC-SVM pe toate datele ar deveni mult prea lent; credit card (~227k rânduri) a fost antrenat pe tot setul, fără subeșantionare.

A.3 Autoencoder Feedforward (MLP)

	CIC flat	CIC context	Yahoo	Credit card
Dimensiune intrare	77	86	64 (fereastră aplatizată)	~30 (PCA)
Dimensiuni ascunse	[64, 32, 16]	[64, 32, 16]	[32, 16]	[16, 8]
Raport bottleneck	0.21	0.19	0.25	~0.27
Activare finală	Sigmoid	Sigmoid	Linear	Linear

Lățimile straturilor sunt scalate în funcție de dimensiunea intrării, astfel încât raportul de compresie al bottleneck-ului să rămână aproximativ constant (~0.20–0.27) pe toate seturile de date. Activarea finală diferă întrucât CIC folosește `MinMaxScaler` (mărginit la $[0, 1]$, potrivit pentru o ieșire sigmoid), în timp ce Yahoo și credit card folosesc scalere nemărginite (ieșire liniară).

A.4 LSTM Predictiv

	CIC	Yahoo	Credit card
INPUT_SIZE	77	1	1
HIDDEN_DIM	128	64	64
NUM_LAYERS	1	1	1
EPOCHS	20	20	50
BATCH_SIZE	256	256	512

HIDDEN_DIM este scalat în funcție de dimensiunea intrării pentru CIC (o extindere de $\sim 1.66\times$ față de cele 77 de caracteristici de intrare); ambele seturi de date univariate folosesc 64, o capacitate suficientă pentru un semnal scalar. Credit card folosește mai multe epoci (50) întrucât curba sa de pierdere continuă să scadă și după epoca 30 – probabil o consecință directă a tratării celor 29 de caracteristici PCA netemporale ca pseudo-secvență, ceea ce face optimizarea mai zgomotoasă decât pe date cu adevărat temporale. CIC și Yahoo converg până la epoca ~ 5 , deci 20 de epoci este deja o alegere conservatoare pentru acestea.

A.5 Autoencoder LSTM

	CIC	Yahoo	Credit card
INPUT_SIZE	77	1	1
HIDDEN_DIM	128	64	64
NUM_LAYERS	1	1	1
EPOCHS	50	30	50
BATCH_SIZE	256	256	512

Atât CIC, cât și credit card au necesitat 50 de epoci pentru a atinge un platou; Yahoo converge până la epoca ~ 5 , deci 30 este deja generos.

A.6 Bottleneck Transformer Autoencoder

	CIC	Yahoo	Credit card
D_MODEL	64	64	64
NHEAD	4	4	4
NUM_LAYERS	2	2	2
DIM_FF	256	256	256
LATENT_DIM	32	32	32
EPOCHS	50	20	20
BATCH_SIZE	256	256	512

LATENT_DIM=32 oferă o compresie uniformă de 50% față de D_MODEL=64, pe toate cele trei seturi de date.

A.7 Causal Transformer

	CIC	Yahoo
D_MODEL	64	64
NHEAD	4	4
NUM_LAYERS	2	2
DIM_FF	256	256
DROPOUT	0.1	0.1
EPOCHS	30	30
BATCH_SIZE	256	256